Univerza v Ljubljani
Fakulteta za računalništvo in informatiko

Klemen Remec

Emulacija računalnika Sinclair ZX Spectrum z vpogledom v stanje sistema

DIPLOMSKO DELO

UNIVERZITETNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE
RAČUNALNIŠTVO IN INFORMATIKA

Mentor: izr. prof. dr. Jurij Mihelič

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.

Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani `http://www.gnu.org/licenses/`.

Besedilo je oblikovano z urejevalnikom besedil LaTeX.

Kandidat: Klemen Remec
Naslov: Emulacija računalnika Sinclair ZX Spectrum z vpogledom v stanje sistema
Vrsta naloge: Diplomska naloga na univerzitetnem programu prve stopnje Računalništvo in informatika
Mentor: izr. prof. dr. Jurij Mihelič

Opis:
Besedilo teme diplomskega dela študent prepiše iz študijskega informacijskega sistema, kamor ga je vnesel mentor. V nekaj stavkih bo opisal, kaj pričakuje od kandidatovega diplomskega dela. Kaj so cilji, kakšne metode naj uporabi, morda bo zapisal tudi ključno literaturo.

Title: Emulation of Sinclair ZX Spectrum computer with system state insight

Description:
opis diplome v angleščini

Na tem mestu zapišite, komu se zahvaljujete za pomoč pri izdelavi diplomske naloge oziroma pri vašem študiju nasploh. Pazite, da ne boste koga pozabili. Utegnil vam bo zameriti. Temu se da izogniti tako, da celotno zahvalo izpustite.

Kazalo

Povzetek

Abstract

1 Uvod **1**

2 ZX Spectrum **3**

2.1 Zgodovina podjetja Sinclair in računalnikov ZX Spectrum . . . 3

2.2 Pregled strojne opreme 6

2.3 Pregled in primerjava z obstoječimi emulatorji 8

2.4 Programiranje emulatorja in razhroščevalnika 9

3 Pomnilnik **13**

3.1 Razdelitev delov pomnilnika in njihov pomen 13

3.2 ROM 14

3.3 Sistemske spremenljivke 15

3.4 Implementacija v emulatorju 15

4 Centralna procesna enota Z80 **19**

4.1 Registri in zastavice 20

4.2 Ukazi 25

4.3 Prekinitve 30

4.4 Implementacija v emulatorju 32

5 ULA **35**
5.1 Časovnik 36
5.2 Zaslon 37
5.3 Tipkovnica 42
5.4 Kasete 48
5.5 Zvok 50

6 Evalvacija emulatorja **53**

7 Zaključek **55**

Članki v revijah **57**

Literatura **59**

Seznam uporabljenih kratic

kratica	angleško	slovensko
ASCII	American Standard Code for Information Interchange	ameriški standardni kod za izmenjavo informacij
BASIC	Beginners' All-purpose Symbolic Instruction Code	programski jezik BASIC
CPE	central processing unit	centralna procesna enota
CRC	cyclic redundancy check	ciklično preverjanje redundance
EAR	earphone	vhod za signal iz kasetofona
IFF	interrupt flip-flop	prekinitveni preklopnik
INT	interrupt	maskirna prekinitev
JVM	Java Virtual Machine	javanski navidezni stroj
KMP	Kotlin Multiplatform	večplatformska tehnologija Kotlin Multiplatform
LIFO	last in, first out	oblika delovanja sklada po principu zadnji noter, prvi ven
MIC	microphone	izhod za signal proti kasetofonu
NMI	non-maskable interrupt	nemaskirna prekinitev
PC	program counter	programski števec
PDF/A	Portable Document Format/Archive	arhivska različica zapisa PDF
RAM	random access memory	bralno-pisalni pomnilnik
ROM	read-only memory	bralni pomnilnik
SNA	snapshot	posnetek stanja emulatorja
SP	stack pointer	kazalec sklada
TAP	tape	zapis standardnih kasetnih blokov
TZX	ZX tape	zapis kasete s časovnimi podatki
ULA	uncommitted logic array	logično vezje ULA
V/I	input/output	vhod/izhod

Povzetek

Naslov: Emulacija računalnika Sinclair ZX Spectrum z vpogledom v stanje sistema

Avtor: Klemen Remec

V vzorcu je predstavljen postopek priprave diplomskega dela z uporabo okolja LaTeX. Vaš povzetek mora sicer vsebovati približno 100 besed, ta tukaj je odločno prekratek. Dober povzetek vključuje: (1) kratek opis obravnavanega problema, (2) kratek opis vašega pristopa za reševanje tega problema in (3) (najbolj uspešen) rezultat ali prispevek diplomske naloge.

Ključne besede: emulacija, Sinclair ZX Spectrum, digitalno ohranjanje.

Abstract

Title: Emulation of Sinclair ZX Spectrum computer with system state insight

Author: Klemen Remec

This sample document presents an approach to typesetting your BSc thesis using LaTeX. A proper abstract should contain around 100 words which makes this one way too short.

Keywords: emulation, Sinclair ZX Spectrum, digital preservation.

Poglavje 1

Uvod

ZX Spectrum, viden na sliki 1.1, je 8-bitni domači računalnik podjetja Sinclair Research, ki je v osemdesetih letih pomembno zaznamoval evropski trg mikroračunalnikov in postal dostopen širokemu krogu domačih uporabnikov.

Slika 1.1: Računalnik ZX Spectrum 48K. [23]

Namen diplomskega dela je razviti delujoč emulator in razhroščevalnik sistema ZX Spectrum 48K. Emulator mora podpirati poganjanje programov napisanih za Spectrum in razhroščevanje z vpogledom v notranje stanje sistema. Zaradi poudarka na razhroščevanje mora pri tem omogočati pregled in spreminjanje vrednosti registrov ter pomnilnika, pregled in razlago pomena

naloženih ukazov in delov pomnilnika. Tak vpogled ni pomemben le za boljše razumevanje sistema, temveč tudi ohranjanje znanja o njegovem delovanju. Delovanje emulatorja se ovrednoti z zagonom nekaj izbranih programov in iger, s čimer potrdimo pravilno delovanje emulacije.

V nadaljnjih poglavjih je najprej predstavljen računalnik ZX Spectrum, njegov vpliv na slovensko in svetovno okolje, in pomen ohranjanja takega znanja. Sledi opis splošne zgradbe emulatorja, zatem pa poglavja o zasnovi in implementaciji posameznih delov sistema, to so pomnilnik, procesor Z80 in ULA, ki obravnava časovnik, zaslon, tipkovnico, kasete in zvok.

Poglavje 2

ZX Spectrum

2.1 Zgodovina podjetja Sinclair in računalnikov ZX Spectrum

Sinclair Research je pred ZX Spectrumom že uveljavil nizkocenovne domače računalnike, kot sta ZX80 in ZX81, ki sta poleg računalnika ZX Spectruma prikazana na sliki 2.1. Bila sta bila namenjena predvsem učenju programiranja in prvi uporabi računalnika doma. ZX Spectrum je bil predstavljen aprila 1982 kot zmogljivejši naslednik ZX81, napram kateremu je na trg prinesel barvno grafiko, višjo ločljivost zaslonske slike, zvok in bolj razvito podporo okolju BASIC [3].

Slika 2.1: Računalniki Sinclair ZX80, ZX81 in ZX Spectrum. [4]

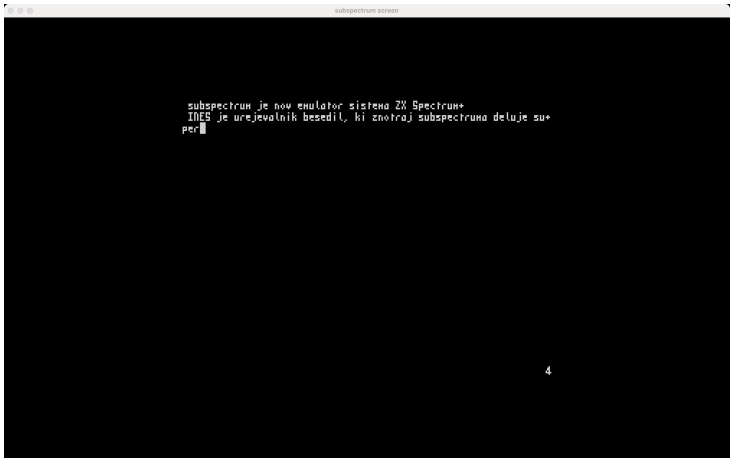
Prvotni Spectrum je bil na voljo v različicah s 16 KB in 48 KB RAMa, obe pa sta uporabljali procesor Zilog Z80 in 16 KB ROMa. Računalnik se je priključil na navaden televizor, programe pa je praviloma nalagal s kasete, kar je pomembno vplivalo na način uporabe in razširjanja programske opreme. Zaradi nizke cene in velike programske knjižnice je Spectrum postal eden najprepoznavnejših evropskih domačih računalnikov osemdesetih let [21] [3].

Družina ZX Spectrum se je pozneje širila z različnimi modeli. Spectrum+ je obdržal tehnično osnovo modela 48K, vendar jo je prestavil v večje ohišje z izboljšano tipkovnico, poznejši Spectrum 128 pa je razširil pomnilnik na 128 KB, dogradil implementacijo okolja BASIC, izboljšal zvočni čip in dodal možnosti za priklop zunanjih naprav. Ker je model 48K predstavljal osrednjo in najbolj razširjeno različico Spectruma ter osnovo za velik del Spectrumove programske opreme, se prav zato nadaljnja obravnava in implementacija emulatorja osredotočata na ta model [3] [14].

2.1.1 Vpliv na slovenski trg mikroračunalnikov in Jakopin

V slovenskem prostoru je bil ZX Spectrum pomemben predvsem kot cenovno dostopen domači računalnik. Prvi Spectrumi so se pri nas pojavili sredi leta 1982, kmalu zatem pa naj bi jih bilo samo v Ljubljani že nekaj tisoč. Njegova priljubljenost je bila povezana z nizko ceno in velikim številom iger, vendar se uporaba ni ustavila le pri zabavi [11].

Takšno okolje je pomembno za razumevanje pomena programa INES Primoža Jakopina. Program ni bil le uporabniški pripomoček, temveč primer doma razvite programske opreme, ki je iz Spectruma, takrat večinoma uporabljanega za igranje iger, naredila delovno orodje. INES je bil januarja 1985 objavljen kot kasetni program, ki je omogočal obdelavo besedil spremeljive dolžine (primer delovanja prikazan na sliki 2.2), slik ter manjših podatkovnih zbirk. Podpora šumnikom in jugoslovanskim črkam je bila posebej pomembna za domačo rabo [11] [18] [7].



Slika 2.2: Prikaz delovanja urejanja besedila v programu INES znotraj razvitega emulatorja.

Tehnično je z INES pokazal, kje so bile postavljene meje učinkovitosti BASICa za uporabo v zahtevnejših programih - iskanje po besedilu je bilo v BASICu bistveno počasnejše napram implementaciji v strojnem jeziku. Ta premik od tolmačenega BASICa proti programiranju v strojnem jeziku se je poznal skozi vso razvojno pot programov od TESS, BESS do INES, ter kasneje programov STEVE za Atari ST in EVA za modernejše osebne računalnike [8] [9] [10] [11].

2.1.2 Pomen ohranjanja računalniških sistemov in programske opreme prek emulacije

Ohranjanje celotne programske dediščine ZX Spectruma in drugih sistemov ni vezano le na shranjeni kasetni zapis programov, temveč tudi na konkretno izvajalno okolje: procesor, pomnilnik, ROM, način nalaganja, tipkovnico, prikaz na zaslonu in druge lastnosti sistema. Digitalno ohranjanje zato zajema tudi računalniške sisteme in programsko opremo kot digitalne artefakte, katerih dostopnost je odvisna od hitro spreminjajočih se nosilcev podatkov, formatov, programov in strojne opreme [16].

Računalniška emulacija pomeni programsko posnemanje drugega računalniškega sistema. Za ohranjanje programov je pomembno posnemanje delovanja notranjih komponent sistema in vhodno-izhodnih naprav, možnost uporabe sistema preko nalaganja programov, vnosa podatkov in prikaza rezultatov. Emulacija tako ohranja digitalni artefakt skupaj z zadostnim delom njegovega izvajalnega in uporabniškega okolja [16].

2.2 Pregled strojne opreme

2.2.1 Kratek pregled strojne opreme

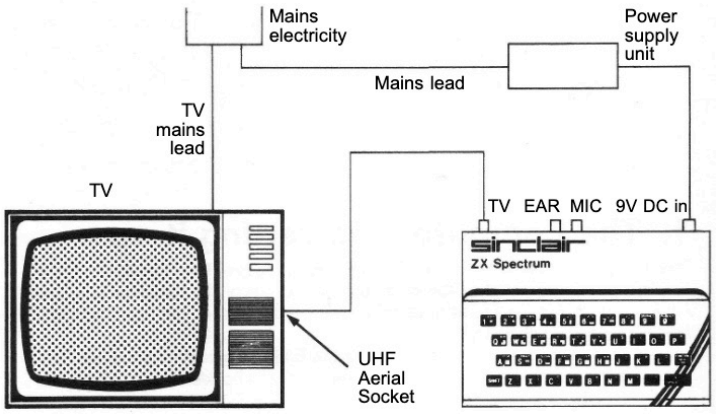
Računalnik ZX Spectrum ima na zadnji strani več vtičnic in priključkov: vtičnico 9V DC IN, ki se prek napajalne enote priključi v električno omrežje, 3,5 milimeterska vtičnica EAR, ki se lahko poveže z izhodno vtičnico za slušalke ali zvočnik na kasetofonu, 3,5 milimeterska vtičnica MIC, ki se poveže z vhodno vtičnico za mikrofon na kasetofonu, vtičnico za televizijo in robni priključek za povezavo z dodatno opremo. Povezave med osnovnimi deli sistema so prikazane na sliki 2.3 [21].

Ko računalnik ugasnemo, se vse v njem shranjene informacije izgubijo, če jih pred tem ne prenesemo na zunanji medij, kot je na primer kaseta. Če se kadarkoli zapletemo pri uporabi računalnika, ga lahko le izključimo iz električnega omrežja in nazaj vklopimo, ter tako ponastavimo brez skrbi poškodbe računalniku [21].

Razporeditev glavnih notranjih komponent računalnika je prikazana na sliki 2.4.

CPE Z80 predstavlja možgane računalnika, ki nadzira vse operacije, izvaja aritmetične račune, sprejema odločitve in na splošno upravlja, kaj naj računalnik naredi [21].

ROM vsebuje 16384 zlogov (16K) navodil v strojnem kodu Z80, ki sestavljajo osnovni program računalnika, in procesorju narekuje, kaj naj stori v vseh predvidljivih okoliščinah. Ta program je edinstven za ZX Spectrum. [21]



Slika 2.3: Skica povezav komponent sistema [21].

RAM obsega osem čipov za shranjevanje podatkov, ki jih procesor želi obdržati, kot so programi v BASICu, spremenljivke, slika za televizijski zaslon in različni elementi, ki spremljajo stanje računalnika. [21].

ULA deluje kot komunikacijski center sistema, ki poskrbi, da se vse, kar zahteva procesor, tudi dejansko izvede, hkrati pa bere pomnilnik za sestavo televizijske slike in pošilja ustrezne signale televizijskemu vmesniku [21].

PAL kodirnik je skupina komponent, ki pretvarja televizijski izhod iz logičnega čipa v obliko, primerno za barvne televizije [21].

Regulator napetosti pretvarja rahlo nestabilno napetost napajanja v popolnoma konstantnih pet voltov za zanesljivo delovanje sistema [21].

UHF/VHF modulator prenaša sliko na televizijski zaslon [21].

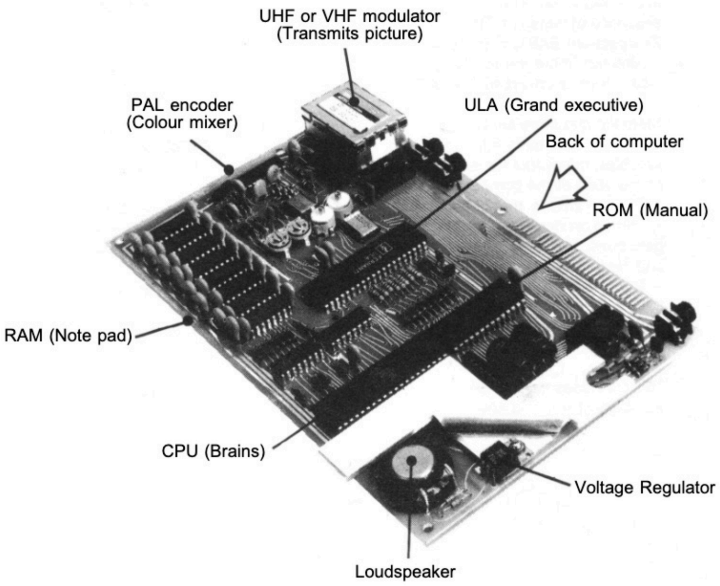
Zvočnik predstavlja zvočni izhod za zvok [21].

2.2.2 Ostala dodatna oprema

Obstaja še druga oprema, ki se lahko uporabi s Spectrumom [20].

ZX Microdrive je naprava za hitro množično shranjevanje in je veliko bolj prilagodljiva pri načinu uporabe kot kasetni snemalnik [20].

Omrežni priključek se uporablja za povezovanje več računalnikov ZX Spectrum, da lahko med seboj komunicirajo [20].



Slika 2.4: Skica notranjih komponent računalnika ZX Spectrum [21].

Vmesnik RS232 omogoča standardno povezavo računalnika s tipkovnicami, tiskalniki, računalniki in drugimi zunanjimi napravami, tudi če niso bili zasnovani posebej za Spectrum [20].

2.3 Pregled in primerjava z obstoječimi emulatorji

Obstaja veliko število emulatorjev računalnika ZX Spectrum, ki se razlikujejo po namenu. Nekateri so namenjeni predvsem udobnemu poganjanju iger in programov, drugi čim natančnejši emulaciji različnih modelov in zunanjih naprav, tretji pa vgradnji emulacije v spletne strani ali razvojnemu delu. Tako je, poleg zmožnosti zagona poljubnega programa, pomembno tudi razlikovanje v podpori različnih modelov računalnika ZX Spectrum, podpora različnim datotečnim formatom, natančnost emulacije ter možnosti vpogleda v notranje stanje sistema.

Med najbolj uveljavljenimi emulatorji sta Fuse in Spectaculator. Fuse

je odprtokodni emulator s podporo vseh večjih operacijskih sistemov in je primer splošnega emulatorja z zelo široko združljivostjo: podpira več modelov Spectruma, sorodne računalnike, številne formate datotek in dodatno opremo. Spectaculator je komercialni emulator, pri katerem je večji poudarek na uporabniški izkušnji: udobnem nalaganju programov, virtualnem kasetofonu, shranjevanju stanja in pripravljenosti za igranje na osebnih računalnikih ter mobilnih napravah. Posebno skupino predstavljajo spletni emulatorji, na primer JSSpeccy 3, kjer je glavna prednost izvajanje neposredno v brskalniku, zato so primerni za enostavnejše predstavitve programov brez namestitve samostojne aplikacije [5] [17] [22].

Obstoječi zreli emulatorji so večinoma namenjeni široki združljivosti, udobni uporabi in podpori velikemu številu modelov, formatov ter dodatnih naprav. Emulator, razvit v okviru tega diplomskega dela, po obsegu ni namenjen neposredni zamenjavi teh rešitev. Omejen je na najpopularnejši model ZX Spectrum 48K in na dele sistema, ki so potrebni za poganjanje tipičnih programov: procesor Z80, pomnilnik, ULA, zaslon, tipkovnico, zvok ter nalaganje kasetnih datotek. Njegova posebnost je pregleden razhroščevalni pregled notranjega stanja sistema, ki ponuja vpogled v vrednosti registrov, pomnilnika in v pomnilnik naloženih razstavljenih ukazov, lažji pregled namembnosti posameznih delov pomnilnika, ROMa in posameznih ukazov znotraj ROMa, izvajanju naloženega programa po korakih, prekinitvenih točkah, in drugih pripomočkov za lažje razumevanje delovanja originalnega sistema.

2.4 Programiranje emulatorja in razhroščevalnika

Emulator je razvit kot aplikacija, ki poleg samega izvajanja programov prikazuje tudi notranje stanje emuliranega računalnika. Zato posamezni deli sistema niso skriti za emulacijsko zanko, ampak so implementirani kot ločeni modeli komponent delujočega sistema: registri, pomnilnik, V/I vrata, časovnik ter ostale naprave in funkcionalnosti računalnika. Procesor vrednosti stanja teh modelov spreminja med izvajanjem ukazov, uporabniški vmesnik pa jih

opazuje in prikaže v razhroščevalniku.

2.4.1 Uporabljene tehnologije

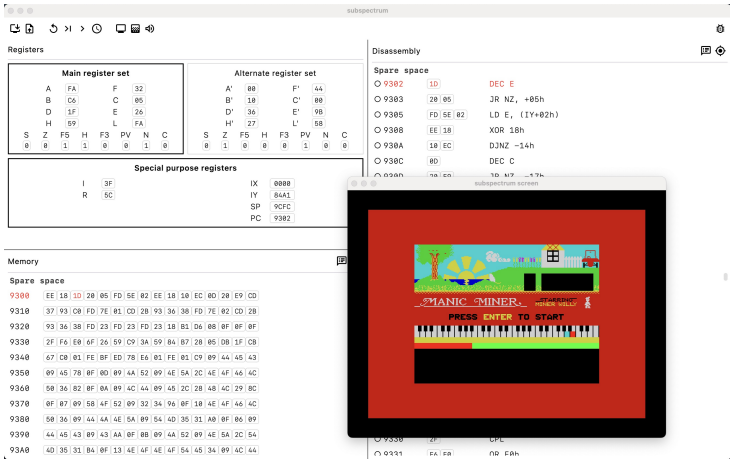
Projekt uporablja ogrodje Kotlin Multiplatform, ki se uporablja za za ustvarjanje aplikacij s skupno kodo za različna izvajalna okolja, na primer namizne, spletne in mobilne aplikacije. V emulatorju se ta zmožnost uporablja predvsem tako, da sta skupna emulacijska logika in večina uporabniškega vmesnika definirana v skupnem modulu, platformno odvisni deli, kot sta izbira datotek in zvočni izhod, pa v ločenih modulih.

Uporabniški vmesnik je narejen s knjižnico Compose Multiplatform. Z njo so implementirana podokna za registre, pomnilnik in razstavljene ukaze, orodna vrstica in ločeno okno zaslona. Daljši tek procesorja se izvaja asinhrono v ozadju, uporabniški vmesnik pa se posodobi le ob spremembah stanja. Gradnjo, izvajanje testov in pripravo namiznih paketov se izvaja preko orodja Gradle.

2.4.2 Kratek pregled delovanja emulacije in vpogleda v sistem

Na sliki 2.5 vidimo primer vmesnika emulatorja. Podokno za registre prikazuje vrednosti splošnin in posebnih registrov ter zastavic, podokno za pomnilnik prikazuje vrednosti pomnilniških lokacij po naslovih, podokno z razstavljenimi ukazi pa prikazuje ukaze, dekodirane iz trenutne vsebine pomnilnika. Grafični izhod emuliranega računalnika je prikazan v ločenem oknu zaslona. Ko procesor ne teče, lahko uporabnik neposredno spremeni vrednosti registrov, pomnilniških celic in ukaznih zlogov.

Iz orodne vrstice lahko uporabnik ponovno naloži ROM ali kasetni program, ponastavi stanje sistema, izvede posamezen procesorski korak, ki izvede trenutni ukaz, zažene in ustavi tek procesorja, v katerem se ti koraki ponavljajo zaustavitve ali prekinitvene točke, preklopi hitrost izvajanja emulacije, odpre okno zaslona ter nastavi način tipkovnice in omogoči zvok.



Slika 2.5: Posnetek zaslona ob uporabi razvitega emulatorja.

Ob zagonu emulator najprej ponastavi stanje sistema in v pomnilnik naloži izbrani ROM. Osrednji del izvajanja je metoda `Processor.step`. Ta prebere ukaz na naslovu programskega števca, ga dekodira, premakne programski števec, izvede učinek ukaza in posodobi vrednost časovnika glede na dolžino izvajanja ukaza.

Poglavje 3

Pomnilnik

Pomnilnik hrani podatke v obliki zlogov (vredosti med 0 in 255), ki se nahajajo na dvozložnih pomnilniških naslovih med `0x0000` in `0xFFFF`. [20]

3.1 Razdelitev delov pomnilnika in njihov pomen

V računalniku ZX Spectrum je pomnilnik razdeljen na ROM, ki se nanaša na naslovni prostor med `0x0000` in `0x3FFF`, ter RAM, ki se nanaša na naslovni prostor med `0x4000` in `0xFFFF` [20].

Branje vrednosti iz pomnilnika deluje pri delu s podatki v ROMu in RAMu, pisanje vrednosti v pomnilnik pa deluje le v RAMu (ukaz pisanja v ROM se bo izvedel, a brez učinka) [13].

RAM je nadalje razdeljen na območja z različnimi nameni uporabe, pomembnejša območja pa so sledeča [20] [13]:

- zaslonska datoteka vsebuje podatke za prikaz vsebine na zaslon,
- atributi položajev znakov hranijo informacije o prikazu zaslona,
- medpomnilnik tiskalnika hrani znake, ki jih mora tiskalnik natisniti, kadar pa tiskalnika ne uporabljamo, ostaja ta prostor neizkoriščen,

- sistemske spremenljivke vsebujejo razne informacije o stanju računalnika,
- sistem BASIC vsebuje uporabniški program v BASICu, njegove spremeljivke in druge podatke ter sklad za aritmetični kalkulator, in se razteza do naslova, na katerega kaže sistemska spremenljivka RAMTOP,
- prosti pomnilnik je zavarovan pred vplivi programa v BASICu in pred posegi nadzornega programa,
- strojni sklad, ki se začenja 2 zloga pod vrhom prostega pomnilnika, uporablja procesor Z80,
- računski sklad predstavlja 30 zlogov velik prostor z začetkom na naslovu, shranjenem v sistemski spremenljivki MEMBOT, ki se uporablja za računske operacije v plavajoči vejici,
- definicije uporabniško definiranih grafičnih znakov.

Pri vnosu več podatkov v dano območje, kot je njegova privzeta velikost, se ostala območja premaknejo navzgor, in obratno pri brisanju podatkov [20].

3.2 ROM

V ROMu shranjen nadzorni program poleg pravilnega zagona opravlja tudi mnoge druge naloge osnovnega delovanja sistema, kot so izrisovanje slike na zaslon, spremljanje pritiskov tipk, tolmačenje pognanega programa v BASICu in posredovanje napak o njegovem izvajanju, delo s plavajočo vejico, itd [21] [13].

Procesor ne pozna sintakse in pravil izvajanja BASICa. Programe v BASICu zato berejo in izvajajo ROMovi podprogrami v strojnem jeziku, strojni programi, tako nadzorni program ROMa kot uporabniški strojni programi, pa se izvajajo neposredno na procesorju [13].

3.3 Sistemske spremenljivke

Zlogi na pomnilniških naslovih med `0x5C00` in `0x5CB5` se uporabljajo za hrambo in spreminjanje informacij o stanju oziroma delovanju sistema [20].

Bolj pomembne sistemske spremenljivke so sledeče [20] [13]:

- VARS na naslovu `0x5C4B`, ki vsebuje naslov začetka spremenljivk programa BASIC,
- PROG na naslovu `0x5C53`, ki vsebuje naslov začetka programa v BASICu,
- E_LINE na naslovu `0x5C59`, ki vsebuje naslov konca programa v BASICu,
- RAMTOP na naslovu `0x5CB2`, ki vsebuje naslov zadnjega zloga sistema BASIC,
- P-RAMT na naslovu `0x5CB4`, ki vsebuje naslov zadnjega zloga fizičnega pomnilnika,
- BORDCR na naslovu `0x5C48`, katerih prvi trije biti definirajo barvo obrobe zaslona,
- ATTR_P na naslovu `0x5C8D`, ki vsebuje definicije uporabljanih barv papirja in papirja, ter drugih atributov stalnega prikaza znakov,
- CHARS na naslovu `0x5C36`, ki vsebuje naslov na začetek nabora znakov.

3.4 Implementacija v emulatorju

Ker model ZX Spectrum 48K ne uporablja preklapljanja pomnilniških bank, je pomnilnik v emulatorju predstavljen kot en sam 64 KB velik naslovni prostor. Pomnilniški naslov je implementiran kot 16-bitna vrednost `UShort`, vsebina pomnilnika pa kot polje zlogov. Polje je ovito v pomožni razred

`DataByteArray`, ki poleg dostopa do posameznih zlogov omogoča tudi kopiranje obsegov in oblikovan heksadecimalni izpis.

Osrednji razred za delo s pomnilnikom je `MemorySet`. Ta vsebuje metode za branje in pisanje ene celice, branje in pisanje zaporednega bloka celic ter branje in nastavljanje posameznega bita. Nad njim je definiran objekt `Memory`, ki vsebuje skupno instanco razreda `MemorySet`. Zato procesor, ULA, nalaganje kaset in uporabniški vmesnik ne hranijo ločenih kopij pomnilnika, ampak vsi dostopajo do istega stanja emuliranega računalnika. Procesor iz njega bere ukazne zloge, operande in podatke, ULA pa iz območij `0x4000-0x57FF` in `0x5800-0x5AFF` bere podatke zaslonske datoteke in atributov.

Meja med ROMom in RAMom je v emulatorju določena s konstanto `ROM_END_ADDRESS`, katere vrednost je `0x3FFF`. Metodi `setMemoryCell` in `setMemoryCells` zato pri običajnem izvajanju ne spremenita celic v območju ROMa. Pri zapisu enega zloga se tak zapis preprosto prezre, pri zapisu večjega bloka pa metoda zapiše samo del bloka, ki pade v območje RAMa. S tem se ohrani obnašanje pravega računalnika, kjer procesor lahko naslovi ROM, zapis vanj pa nima učinka.

Blokovni zapis preveri še dve napaki: ali je zapisani blok večji od celotnega pomnilnika in ali bi se zapis raztezal čez naslov `0xFFFF`. To je pomembno pri nalaganju ROMa, kasetnih blokov in testnih programov, saj se napačen naslov ali napačna dolžina podatkov odkrije že ob poskusu zapisa. Branje obsega deluje prek metode `getMemoryCells`, ki vrne kopijo zahtevanega dela pomnilnika, zato klicatelj ne more mimo razreda `MemorySet` spreminjati notranjega polja.

Ob ponastavitvi emulator najprej ustavi procesor, ponastavi registre, V/I vrata, časovno stanje ULA in stanje kasetnega pogona, nato pa razred `MemorySet` celoten pomnilnik napolni z vrednostjo `0x00`. Metoda `loadRom` po ponastavitvi prebere datoteko ROMa za model 48K in jo zapiše od naslova `0x0000` naprej. Isti mehanizem blokovnega zapisa se uporablja tudi pri hitrem nalaganju vsebine izbrane datoteke posnetka kasete ob ukazu `LOAD`, blokovno branje pomnilnika pri ukazu `VERIFY` pa enak obseg iz pomnilnika

prebere in ga primerja z vsebino kasetnega bloka.

Podokno razhroščevalnika za pregled pomnilnika prikazuje začetni naslov vrstice, vrednosti zlogov in trenutni položaj programskega števca. Podokno za razstavljene ukaze iz istega pomnilnika sproti dekodira ukaze procesorja Z80 in prikaže naslov, pripadajoče zloge, razstavljeno obliko ukaza, trenutni položaj programskega števca in prekinitvene točke.

Vsak zapis v pomnilnik sproži obvestilo o spremembi vsebine prek toka `SharedFlow`. Obe podokni ta obvestila zbirata in prikaz osvežita ob naslednjem izrisu. Ker se zaporedna obvestila združujejo, hitro izvajanje programa ne povzroči ločenega preračunavanja uporabniškega vmesnika ob vsaki spremembi vsebine poljubne pomnilniške celice.

Poleg običajnega heksadecimalnega prikaza razhroščevalnik podpira še anotiran pogled. Stalna območja RAMa, kot so ROM, zaslonska datoteka, atributi, medpomnilnik tiskalnika in sistemske spremenljivke, so zapisana v registru območij, meje dinamičnih območij pa se izračunajo iz vrednosti določenih sistemskih spremeljivk ter skladovnega kazalca `SP`. Zato lahko vmesnik predstavitev teh območij prilagodi trenutnemu programu v BASICu, njegovim spremenljivkam, delovnemu prostoru, računskemu skladu, prostemu pomnilniku, strojnemu skladu in uporabniško definiranim grafičnim znakom.

Imena in opisi označenih naslovov temeljijo na anotirani razčlembi stalnih območij RAMa, podprogramov v ROMu in sistemskih spremenljivk. Iz teh virov so povzeti začetni naslovi, meje območij in kratki opisi njihove vloge v sistemu [15] [19].

Ko je procesor zaustavljen, ima uporabnik v teh podoknih možnost neposredno urejati vsebino pomnilniških celic, kar velja tudi za območje pomnilnika z ROMom, na katerega navadno izvajanje uporabniškega programa sicer nima vpliva.

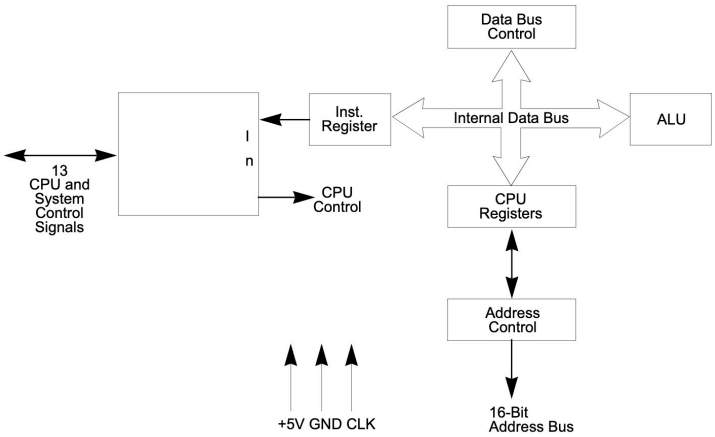
Poglavje 4

Centralna procesna enota Z80

Procesor Z80 najdemo v mnogih računalnikih - Spectrum, ZX 81, Galaksija, Dialog 20 Gorenje, Amstrad CPC 454 in celo večji računalniki, kot je Iskrin Partner [13].

Družina komponent procesorjev Z80 podjetja Zilog je četrta generacija mikroprocesorjev, ki dosega hitrosti od 6 do 20 MHz. Čeprav ima mnogo 8-bitnih in nekaj 16-bitnih registrov, velja za 8-bitni procesor in večino časa obdeluje 8-bitne podatke v 8-bitnih registrih in 8-bitnih pomnilniških celicah [13] [28].

Glavni notranji elementi procesorja so prikazani na sliki 4.1. Vsak sistem s procesorjem Z80 mora vsebovati določene zunanje strojne elemente, kot so vir napajanja +5V, oscilator, pomnilniške naprave (RAM, ROM ali PROM) in vhodno-izhodna vezja, katerih signali so časovno usklajeni s procesorjem za nadzor pomnilnika ali perifernih vezij. [28]



Slika 4.1: Diagram notranje arhitekture in glavnih elementov procesorja Z80 [28].

4.1 Registri in zastavice

4.1.1 Razporeditev registrov, njihove velikosti in pomen

Večina operacij Z80 je 8-bitnih, torej dela z vrednostmi med `0x00` in `0xFF`, kar lahko predstavlja bodisi nepredznačen obseg vrednosti od 0 do 255, bodisi predznačen obseg vrednosti od -128 do 127 [13].

Nekatere operacije pa so 16-bitne, torej CPE dela z vrednosti med `0x0000` in `0xFFFF`, kar lahko predstavlja bodisi nepredznačen obseg vrednosti od 0 do 65535, bodisi predznačen obseg vrednosti od -32768 do 32767. 16-bitna števila največkrat potrebujemo za naslavljanje pomnilniških celic [13].

Posebnost so vrednosti števil v obliki plavajoče vejice, ki zavzemajo kar 5 zlogov (prvi zlog vsebuje $128 + n$, ostali 4 zlogi vsebujejo mantiso z vrednostjo $[\frac{1}{2}, 1)$). Ker 5-zložnih vrednosti ne moremo shraniti v posamezen register ali registrski par, za njihovo shranjevanje uporabljamo bodisi celoten registrski petorček AFBCDE, bodisi računski sklad v pomnilniku. Vender pa tak tip podatka ni specifičen za procesor Z80, temveč za podprograme ROMa

računalnika ZX Spectrum [13].

Procesor Z80 vsebuje 208 bitov bralno-pisalnega pomnilnika, implementiranega kot statični RAM, ki predstavlja 18 8-bitnih registrov in 4 16-bitne registre [28].

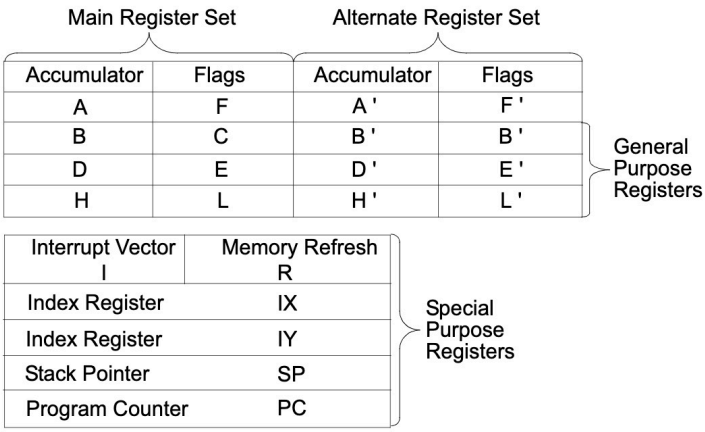
Ti notranji registri se delijo na dva registrska nabora, od katerih vsak vsebuje 6 splošnonamenskih registrov (registri B, C, D, H in L), akumulator (register A) in register zastavic (register F), ter 6 posebnonamenskih registrov, ki so 16-bitni programski števec (register PC), 16-bitni kazalec sklada (register SP), 16-bitna indeksna registra IX in IY, osvežitveni register R in register naslova strani prekinitev I. Razporeditev notranjih registrov je prikazana na sliki 4.2 [13] [28].

Pri tem lahko splošnonamenske 8-bitne registre uporabljamo posamično, ali pa pare 8-bitnih registrov združimo v 16-bitne registrske pare BC, DE in HL [13] [28].

Med osnovnim in zamenljivim registrskim naborom (keterega imena registrov označujemo z apostrofom ob imenih) se lahko menjava. Ta značilnost procesorja Z80 je posebej potrebna v sistemih, ki zahtevajo hiter odziv na prekinitve. Ob pričetku obdelave prekinitev ali izvajanja podprograma je potrebno star izvajalni kontekst (vrednosti registrov) shraniti do obnovitve pred vrnitvijo iz podprograma. Zamenljiva registrska nabora tako omogočata mnogo hitrejšo menjavo konteksta napram hrambi in obnavaljanju vrednosti vseh registrov v pomnilniku [13] [28].

Akumulator oz. register A velja za edini register, kjer procesor lahko opravlja 8-bitne aritmetično-logične operacije. Pri njihovem izvajanju register zastavic F v splošnem hrani posebne pogoje izvedenih operacij ali značilnosti dobljenega rezultata [13] [28].

Register naslova strani prekinitev (I) hrani višji zlog naslova prekinitveno-servisnega programa, ki se dopolnejo z nižjim zlogom naslova, ki jih zagotovi zunanja naprava, ki je sprožila prekinitev. ko naprava, ki povzroči prekinitev. Procesor Z80 namreč deluje v načinu, v katerem je mogoče v odziv na prekinitev izvesti posredni klic na PSP na poljubnem mestu v pomnilniku



Slika 4.2: Diagram notranjih registrov procesorja Z80 [28].

[28].

Register osveževanja pomnilnika (R) hrani 7-bitni števec osveževanja pomnilnika, ki se samodejno poveča po vsakem pridobljenem ukazu, osmi bit se uporablja le pri ukazu `LD R, A`. To omogoča enostavno uporabo dinamičnih in statičnih pomnilnikov brez upočasnjevanja delovanja CPE (osveževanje se dogaja med dekodiranjem prej pridobljenega ukaza) [28].

Za 16-bitne registre in registrske pare velja, da je višji zlog shranjen v registru/registrskem paru prvi, nižji pa drugi. Pri tem je potrebno opozoriti, da je tak vrstni red ravno obraten kot pri hranjenju 16-bitne vrednosti v pomnilniku, kjer prva pomnilniška celica vsebuje nižji del vrednosti, druga pa višjega [13] [28].

Indeksna registra IX in IY hranita 16-bitni naslov, ki se uporablja v indeksnem načinu naslavljanja kot bazni kazalec, iz katerega po prištetju odmika (dodaten zlog, predstavljen kot število v dvojiškem komplementu), dobimo končni pomnilniški naslov operanda [28].

Programski števec PC hrani naslov trenutnega ukaza, ki se pridobiva iz pomnilnika, in se samodejno poveča, ko se njegova vsebina prenese na naslovno vodilo. PC se uporablja tudi za klice in vračanje iz podprogramov, pri čemer se povratni naslov hrani na skladu v pomnilniku [13] [28].

Kazalec sklada SP hrani naslov vrhnje vrednosti na skladu. Sklad, organiziran v obliki LIFO, omogoča preprosto izvajanje večstopenjskih prekinitev, teoretično neomejeno gnezdenje podprogramov in poenostavitev mnogih vrst manipulacije podatkov [13] [28].

4.1.2 Zastavice

Zastavice, hranjene v registrih F in F', predstavljajo informacijo o stanju bodisi vsebine registra A, ali o ravnokar opravljeni računski operaciji. Kljub temu, da ima register F možnost shranjevanja 8 bitov, za dokumentirane zastavice uporablja le 6 bitov (bita 3 in 5 se ne uporabljata) [13] [28]:

- zastavica znaka S (angl. Sign flag) na bitu 7 je dvignjena, ko je rezultat operacije negativen,
- ničelna zastavico Z (angl. Zero flag) na bitu 6 je dvignjena, ko je rezultat operacije enak nič,
- zastavica polprenosa H (angl. Half carry flag) na bitu 4 je dvignjena, ko se je v zadnji operaciji spremenil 5. bit registra,
- zastavica parnosti/prepolnjenja P/V (angl. Parity/oVerflow flag) na bitu 2 je dvignjena, ko je v rezultatu operacije parno število bitov ali ko se je spremenil 8. bit registra (kar pomeni možnost preliva),
- zastavica odštevanja N (angl. subtract flag) na bitu 1 je dvignjena, ko je zadnja operacija bila odštevanje,
- zastavica prenosa C (angl. Carry flag) na bitu 0 je dvignjena, ko se je v zadnji operaciji zgodil prenos.

Zastavice omogočajo vejanje programa prek pogojnih skokov z možnimi pogoji [13]:

- Z (angl. Zero) ob dvignjeni ničelni zastavici,

- NZ (angl. Not Zero) ob spuščeni ničelni zastavici,
- C (angl. Carry) ob dvignjeni zastavici prenosa,
- NC (angl. Not Carry) ob spuščeni zastavici prenosa,
- M (angl. Minus) ob dvignjeni zastavici znaka,
- P (angl. Positive) ob spuščeni zastavici znaka,
- PE (angl. Parity Even) ob dvignjeni zastavici parnosti,
- PO (angl. Parity Odd) ob spuščeni zastavici parnosti.

4.1.3 Nedokumentirane zastavice

Uradna dokumentacija procesorja Z80 bita 5 in 3 registrov zastavic F in F', ki ju bomo označevali kot YF in XF, pogosto obravnava kot neuporabljena, vendar ju dejanski procesor vseeno nastavlja in uporablja. Ti zastavici nista pogojni zastavici v enakem smislu kot ničelna zastavica Z ali zastavica prenosa C, saj Z80 nima pogojnih skokov, klicev ali vrnitev, ki bi neposredno preverjali YF ali XF. Zaradi tega se zdita bita za običajno programiranje nepomembna, pri emulaciji pa sta pomembna za programe, ki uporabljajo stranske učinke ukazov. Zato ju emulator, ki želi natančno posnemati procesor, kljub pomanjkljivi uradni dokumentaciji ne sme obravnavati kot neuporabljena [27].

Pri običajnih aritmetično-logičnih ukazih z 8-bitnim rezultatom se YF in XF praviloma nastavita iz bitov 5 in 3 tega rezultata. Pri ukazih primerjave se rezultat ne shrani v akumulator, zato se YF in XF nastavita iz bitov 5 in 3 primerjanega operanda. Pri 16-bitni aritmetiki se YF in XF nastavita tako, kot da bi ju določila operacija nad višjima zlogoma operandov. Pri ukazih preverjanja stanja posameznega bita se YF nastavi, kadar ukaz preverja bit 5 in je ta bit postavljen, XF pa kadar preverja bit 3 in je ta bit postavljen. A to ne velja pri indeksnem naslavljanju, kjer se YF in XF nastavita iz bitov 5 in 3 višjega zloga efektivnega naslova indeksiranega naslavljanja, ter pri

ukazu `BIT n,(HL)`, kjer se YF in XF nastavita iz vrednosti nedokumentiranega notranjega registra, povezanega z nekaterimi predhodnimi 16-bitnimi aritmetičnimi ukazi ter uporabljenimi efektivnimi naslovi. Pri bločnih ukazih prenosov in primerjav se YF in XF nastavita iz vmesnih vrednosti prenosov ali izračunov primerjav. Pri bločnih vhodno-izhodnih ukazih se po prenosu zloga najprej zmanjša register B, nato pa se YF nastavi iz bita 5 nove vrednosti registra B, XF pa iz njenega bita 3 [27].

4.2 Ukazi

ZX Spectrum omogoča uporabo visokonivojskega jezika BASIC. Jezik je bil leta 1964 razvit na Dartmouth College v New Hampshiru, in je bil v svojem času zelo razširjen na osebnih računalnikih. Ima več različic, med katerimi obstajajo manjše razlike, kot na primer zahteva po zapisu ukazne besede LET pri dodeljevanju vrednosti spremenljivkam, kar v mnogih drugih različicah jezika BASIC lahko opustimo. A različica BASICa za ZX Spectrum vseeno ni daleč od (neobstoječega standarda) jezika BASIC, zato ne bi smeli imeti preveč težav pri prilagajanju katerega koli BASIC programa za delovanje na ZX Spectrum [21].

Za programiranje na Spectrumu lahko uporabljamo tudi strojni jezik, ki v večini primerov omogoča hitrejše izvajanje, učinkovitejšo izrabo pomnilnika, krajše programe (za hrambo v pomnilniku omejene velikosti) ter neodvisnost od nadzornega programa, ki je potreben za izvajanje programov v BASICu. A prinaša tudi nekatere slabosti, kot so slabša berljivost programov, omejenost na izvajanje programa v sistemih z istim procesorjem, ter nasploh daljši programi (za hrambo v pomnilniku) [13].

Berljivost strojnega programa lahko izboljšamo s pisanjem v zbirnem jeziku, ki predstavlja preslikavo strojnega jezika v ljudem razumljivejšo obliko, v kateri vsak zbirni ukaz ustreza enemu strojnemu ukazu (in obratno). Zbirni jezik je tako razumljivejši ljudem, a ga moramo pred izvajanjem prevesti v zaporedje zlogov strojnega koda, bodisi ročno, bodisi z uporabo zbirnika.

Najbolj razširjen zbirnik za uporabo v Spectrumu je bil program GENS [20] [13].

4.2.1 Število ukazov in sklopi, njihove dolžine

Procesor Z80 lahko izvaja 158 različnih (dokumentiranih) ukazov, vključno z vsemi 78 ukazi procesorja 8080A [28].

Ukaze delimo na več skupin [28]:

- ukaze nalaganja in izmenjav, ki premikajo podatke med izvorno in ciljno lokacijo, bodisi med posameznimi registri ali registri in pomnilnikom,
- aritmetične in logične ukaze, ki jih procesor izvaja nad podatki v akumulatorju in drugih splošnonamenskih registrih ali pomnilniških lokacijah. Rezultat dane operacije se razen v par izjemah shrani v akumulator, glede na rezultat operacije pa se nastavijo tudi ustrezne zastavice v registru zastavic,
- ukaze rotacije in zamikov, ki omogočajo levo ali desno, aritmetično ali logično, rotacijo ali zamik vrednosti poljubnega registra ali pomnilniške lokacije z ali brez prenosa,
- ukaze manipulacije bitov, ki omogočajo nastavitev, ponastavitev ali preveritev vrednosti poljubnega bita vrednosti v akumulatorju, drugem splošnonamenskem registru ali pomnilniški lokaciji,
- ukaze skokov, klicev in vrnitev, ki se uporabljajo za skoke med več lokacijami v programu. Edinstvena vrsta klicev so ukazi skupine ponovnih zagonov, ki lahko kot nov naslov izvajanja določi le enega od 8 ločenih naslovov, ki se nahajajo na prvi strani pomnilnika,
- ukaze prenosov in iskanja blokov, ki se uporabljajo za prenos celih blokov podatkov iz enega mesta pomnilnika na drugo oziroma iskanje poljubnega 8-bitnega znaka v bloku podatkov v pomnilniku. Ob dosegu konca definiranega bloka ali najdenega znaka se ukaz samodejno

prekine, prav tako se lahko izvajanje takega ukaza kadarkoli prekine in kasneje nadaljuje, kar omogoča vmesno uporabo procesorja za druge operacije. Ukazi te skupine pridejo posebaj prav pri pri obdelavi velikih nizov podatkov,

- ukaze vhoda in izhoda, ki omogočajo prenose podatkov (posameznih ali blokov) med pomnilniškimi lokacijami ali splošnimi registri procesorja in zunanjimi V/I napravami,

- nadzorni oziroma kontrolni ukazi, ki omogočajo upravljanje različnih načinov delovanja procesorja, med drugim omogočanje in onemogočanje prekinitev, nastavljanje načina odziva na prekinitve, itd.

Podskupina ukazov ponovnih zagonov RST (angl. ReSTart) opravlja isto nalogo kot brezpogojni klici, le da lahko z njimi kličemo le 8 naslovov v začetku ROMa [13]:

- RST `0x00` učinkuje kot izklop in ponovnen vklop računalnika,

- RST `0x08` služi za prikaz sporočila o napaki BASICa, katere 8-bitni kod napake sledi ukazu kot operand, ter za uporabljanje mikrotračnih enot in omrežja Spectrumov preko vmesnika na zadnji strani računalnika,

- RST `0x10` na zaslon izpiše znak, katerega kod je v registru A,

- RST `0x18` register A napolni z vrednostjo zloga, katerega naslov je v sistemski spremenljivki CH-ADD,

- RST `0x20` se uporablja pri tolmačenju programov v BASICu,

- RST `0x28` se uporablja za računanje s plavajočo vejico na računskem skladu, zlogi operandi pa določajo operacije, ki se morajo izvesti,

- RST `0x30` preuredi delovni pomnilniški prostor tako, da dobimo v njem toliko prostih zlogov, kolikor je vrednost registrskega para BC,

- RST `0x38` preleti tipkovnico (kontrola pritisnjenih tipk) in poveča števec časa.

Str. 132-133 - kodi operacij za ukaz RST 28 in tabele podprogramov [13]

4.2.2 Tipi naslavljanj

Naslavljanje se nanaša na način definicije naslova podatkov v registrih, pomnilniku ali V/I vratih, ki se bodo uporabili pri izvedbi določenega ukaza [28].

Registrsko naslavljanje (oblike `LD r, r`) določi enega ali več registrov, ki se uporabijo za dano operacijo [13] [28].

Sprotno ali takojšnje naslavljanje (oblike `LD r, n`) v zlogu, ki sledi št. strojnega ukaza, vsebuje 8-bitno vrednost operanda [13] [28].

Razširjeno takojšnje naslavljanje (oblike `LD rr, nn`) v zlogih, ki sledita št. strojnega ukaza, vsebuje 16-bitno vrednost operanda [28].

Zunanje ali razširjeno naslavljanje (oblik `LD A, (nn)` in `LD (nn), A)`) v zlogih, ki sledita št. strojnega ukaza, vsebuje pomnilniški naslov operanda ali skočne lokacije, kar omogoča prenašanje vrednosti iz registra v pomnilniško celico in obratno ter skok ali prenos podatkov med poljubnima pomnilniškima lokacijama [13] [28].

Registrsko posredno naslavljanje (oblik `LD r, (rr)` in `LD (rr), r`) določi registrski par, katerega vrednost predstavlja pomnilniški naslov operanda [13] [28].

Relativno naslavljanje v zlogu, ki sledi št. strojnega ukaza, vsebuje odmik v obliki dvojiškega komplementa, ki se prišteje vrednosti registra PC za določitev pomnilniške lokacije operanda. To poleg prenosljivosti programa omogoča tudi skoke na bližje pomnilniške lokacije z le dvozložnim ukazom [28].

Indeksno naslavljanje (oblik `LD r, (IX+dis)` in `LD (IX+dis), r`) je le oblika registskega posrednega naslavljanja z dodanim odmikom. V zlogu,

ki sledi št. strojnega ukaza, vsebuje odmik v predstavitvi dvojiškega komplementa, ki se prišteje vrednosti izbranega indeksnega registra za določitev pomnilniške lokacije operanda. To poleg prenosljivosti programa omogoča tudi poenostavitev dela s tabelami podatkov [13] [28].

Bitno naslavljanje s pomočjo registrskega, registskega posrednega ali indeksnega naslavljanja lahko določi poljubni bit poljubne pomnilniška lokacije ali registra za izvedbo bitne operacije [28].

Spremenjeno naslavljanje ničelne strani se nanaša na posebne enozložne ukaze RST za skok na poljubno od 8 lokacij v strani 0 pomnilnika, kjer se nahajajo pogosto uporabljeni podprogrami ROMa [28].

Implicitno naslavljanje se uporablja pri mnogih ukazih, ki implicitno določijo registre z operandi, potrebnimi za dano operacijo (npr. rezultat aritmetičnih operacij se implicitno shrani v akumulator) [28].

Ker mnoge operacije zahtevajo več kot en operand, se lahko hkrati uporabi tudi več tipov naslavljanj [28].

4.2.3 Nedokumentirani ukazi

Poleg uradno navedenega nabora ukazov ima Z80 tudi množico operacijskih kod, ki so v uradni dokumentaciji izpuščene ali označene kot neuporabljene. Na pravem procesorju te operacijske kode kljub temu lahko izvedejo nove ali podvojene že dokumentirane ukaze. To se zgodi zato, ker procesor tudi pri nedokumentiranih kombinacijah bitov uporabi isto dekodirno logiko kot pri uradnem naboru ukazov. Predpona določi skupino ukazov, naslednji zlogi pa določijo operacijo, register ali način naslavljanja [27].

Pri predponi `0xCB` nedokumentirane kode od `0xCB30` do `0xCB37` med drugim definirajo ukaze logičnih pomikov v levo `SLL`. Pri predponi `0xDD` se uporabe registra HL v ukazih praviloma nadomestijo z IX, pri predponi `0xFD` pa z IY, vključno z dostopi do posameznih polovic oziroma zlogov indeksnih registrov. Pri predponi `0xED` nekatere kode predstavljajo dvojnike uradnih ukazov, preostale neuporabljene kode pa se obnašajo kot zaporedja brez učinka. Posebnost sta ukazni kodi `0xED70` in `0xED71`, od katerih prva prebere vre-

dnost iz V/I vrat in z njo nastavi samo zastavice, druga pa na V/I vrata zapiše vrednost nič. Pri štirizložnih skupinah s predponama `0xDD 0xCB` in `0xFD 0xCB` se bitne operacije izvedejo nad indeksirano pomnilniško lokacijo, pri delu ukazov pa zadnji zlog določi še dodatni ciljni register [27].

4.3 Prekinitve

Prekinitve omogočajo prekinitev operacij CPE, izvajanja PSP in vrnitev v prvotno stanje izvajanje programa. Prekinitev lahko sproži zunanja naprava ali nadzorni program računalnika. Na primer, CPE vsako 1/50 sekunde pogleda stanje posebne notranje zastavice IFF1. Če je njena vrednost 1, sproži prekinitev in izvrši ukaz RST 38H, s katerim pridobi stanje tipkovnice [13] [28].

Ker se zaradi tega delovna hitrost procesorja nekoliko zmanjša, lahko program z ukazom DI prekinitve onemogoči in kasneje z ukazom EI nazaj omogoči. Program lahko prav tako ukazom HALT prekine izvajanje strojnega programa do naslednje prekinitve. Med tem CPE izvaja operacije NOP za namen delovanja osveževanja pomnilnika [13] [28].

4.3.1 Načini prekinitev in njihovo delovanje

Z80 CPE podpira 2 vrsti prekinitev:

Maksabilne prekinitve INT, ki jih je mogoče programsko onemogočiti, z nastavljanjem vrednosti IFF1 prek ukazov EI in DI, saj se uporablja kot zahteva za prekinitev s strani zunanjih naprav. CPE ima poleg notranje zastavice IFF1, ki se dejansko uporablja pri izvajanju, še IFF2 kot začasno shrambo vrednosti IFF1. Glavna stanja obeh notranjih zastavic in spremembe ob pomembnih ukazih so prikazane v tabeli 4.1 [28].

CPE je lahko nastavljen v enega izmed 3 možnih načinov obravnavanja maskabilnih prekinitev [28].

V načinu 0 lahko prekinitvena naprava na podatkovno vodilo postavi poljuben ukaz, ki ga CPE izvede. Ker mora izbran ukaz imeti le 1 zlog, je to

Dejanje	**IFF1**	**IFF2**	**Opombe**
Ponastavitev CPE	0	0	Prekinitve so onemogočene.
Izvedba ukaza `DI`	0	0	Prekinitve so onemogočene.
Izvedba ukaza `EI`	1	1	Prekinitve so omogočene.
Izvedba ukaza `LD A,I`	*	*	Vrednost IFF2 se shrani v zastavico paritete.
Izvedba ukaza `LD A,R`	*	*	Vrednost IFF2 se shrani v zastavico paritete.
Sprejem NMI	0	*	Prekinitve so onemogočene.
Izvedba ukaza `RETN`	IFF2	*	Dvignjena IFF2 označi konec rutine za obdelavo nemaskirne prekinitve.

Tabela 4.1: Tabela stanj in sprememb stanj IFF1 in IFF2 [28].

pogosto ukaz klica PSP. CPE pri izvedbi tega ukaza samodejno doda 2 čakalni periodi, da bi zagotovil dovolj časa za izvedbo zunanje verižne povezave za nadzor prioritet [28].

V načinu 1 CPE začne izvajati ukaz na pomnilniškem naslovu `0x0038`. Posledično je odziv razen lokacije klica identičen odzivu na nemaskabilne prekinitve, ki kliče podprogram na pomnilniškem naslovu `0x0066`. CPE tudi v tem primeru doda 2 čakalni periodi [28].

Način 2 omogoča klic podprograma v uporabniško vzdrževani tabeli 16-bitnih začetnih naslovov PSP na poljubnem mestu v pomnilniku. Naslovi PSP v tabeli se morajo nahajati na sodih naslovih, ker vsak zavzema 2 zloga. Zgornji zlog 16-bitnega naslova PSP določa vrednost registra I, spodnji zlog pa sama prekinitvena naprava. Ko prekinitvena naprava zagotovi spodnji del kazalca, CPE samodejno potisne PC na sklad, pridobi naslov PSP iz tabele in izvede skok nanj. Ta način odziva zahteva 19 čakalnih period [28].

Prekinitve NMI imajo višjo prioriteto kot INT in jih ni mogoče programsko onemogočiti, saj se uporablja za predvsem za takojšnje odzive na naj-

pomembnejše signale, ki se morajo obravnavati s čim manj zakasnitve. CPE ga zato obravnava že po koncu izvajanja trenutnega ukaza. Pri tem se najprej IFF1 onemogoči (in njegova prejšnja vrednost shrani v IFF2), kar za čas izvajanja PSP prepreči navadne prekinitve, nato se vrednost registra PC shrani na sklad, izvajanje pa preusmeri na pomnilniški naslov `0x0066` [28].

4.4 Implementacija v emulatorju

Procesor je v emulatorju predstavljen z objektom `Processor`. Ta ne hrani vseh podatkov procesorja neposredno v sebi, ampak povezuje ločene modele registrov, pomnilnika, V/I vrat in časovnika. V objektu `Processor` so shranjena predvsem stanja, ki vodijo izvajanje, kot so omogočenost maskirnih prekinitev `IFF1` in `IFF2`, trenutni prekinitveni način, stanje `HALT`, ter funkcionalnosti razhroščevalnika zastavica teka, prekinitvene točke in števec izvedenih ukazov.

Registri so zbrani v objektu `Registers`. Ta vsebuje glavni in zamenljiv registrski nabor ter posebne registre `I`, `R`, indeksna registra `IX` in `IY`, `SP`, `PC` in nedokumentirani notranji register `MEMPTR`. Zastavice niso ločen podatkovni tip, ampak posamezni biti v registru `F`. Dostop do njih je zato implementiran z metodami za branje in nastavljanje posameznih bitov.

Posamezen strojni ukaz je predstavljen z vmesnikom `Instruction`. Ukaz pozna naslov, zloge, iz katerih je bil dekodiran, število T-stanj in metodo `execute`, ki izvede vse potrebne spremembe na stanju sistema ob izvedbi. Opis ukaza je ločen od njegove izvedbe z vmesnikom `InstructionDefinition`. Definicija vsebuje bitni vzorec ukaznega zloga oziroma zaporedja zlogov in metodo za dekodiranje operandov.

Bitni vzorci so opisani z razredom `BitPattern`. V njem so fiksni biti zapisani kot ničle in enice, ostali znaki pa predstavljajo zajeta polja, na primer kodo registra, pogoj skoka, neposredni zlog ali 16-bitni operand. Dekodirnik v objektu `Instructions` iz pomnilnika prebere toliko zlogov, kot jih zahteva posamezen vzorec, prebrano vrednost primerja z registriranimi vzorci in ob

ujemanju ustvari konkreten objekt ukaza.

Posebna obravnava je potrebna pri predponah kodov ukazov. Predponi `DD` in `FD` pomenita, da se nekateri dostopi do `HL`, `H` in `L` začasno preusmerijo na `IX` oziroma `IY`. To je v emulatorju izvedeno z začasnim načinom indeksne predpone v objektu `Registers`. Dekodirnik zna obravnavati tudi kombinacije s predponama `CB` in `ED` ter več zaporednih indeksnih predpon. Tak ukaz se ovije v pomožni objekt, ki ohrani izvorne zloge, prišteje dodatna T-stanja za predpone in nastavi pravilen način dostopa do indeksnih registrov le za čas izvedbe takega ukaza.

Izvajanje enega koraka (naslednjega ukaza) se začne z branjem trenutnega programskega števca `PC`. Če je na tem naslovu nastavljena prekinitvena točka, se neprekinjen tek ustavi. Nato ima prednost morebitna nemaskirna prekinitev. Če je ni, se preverijo še pogoji za hitro nalaganje kasetnih podatkov. Šele nato se ukaz dekodira iz pomnilnika, števec osveževanja `R` se poveča za število dejansko prebranih ukaznih zlogov, programski števec pa se pomakne za dolžino ukaza.

Po predpripravi ukaza se izvede metoda `execute`. Ta lahko spremeni registre, zapisuje v pomnilnik, bere ali piše V/I vrata. Po izvedbi se poveča števec izvedenih ukazov za namene razhroščevalnika, časovniku ULA pa se prišteje število T-stanj ukaza. Pogoji za izvedbo maskirne prekinitve se preverjajo ob koncu izvajanja trenutnega ukaza.

Pri vseh maskirnih prekinitvah se trenutna vrednost registra `PC` najprej zapiše na sklad, nato se onemogočita `IFF1` in `IFF2`. Nemaskirna prekinitev ponastavi vrednost le `IFF1`, na sklad shrani vrednost registra `PC` in nadaljuje izvajanje na naslovu `0x0066`. Ukaza `EI` in `DI` nastavita še notranjo zastavico, zaradi katere se maskirna prekinitev ne obdela takoj za njima.

Ukaz `HALT` je izveden tako, da programski števec vrne na naslov ukaza `HALT` in nastavi stanje zaustavitve. Dokler ne pride do prekinitve, se zato ob vsakem koraku znova izvaja isti ukaz, v tem primeru ukaz `NOP`. Ob prekinitvi se stanje `HALT` zaključi, na sklad pa se zapiše naslov naslednjega ukaza, kot pri pravem procesorju.

Poleg izvajanja ukazov po korakih emulator podpira tudi neprekinjen tek. V tem načinu delovanja procesor ponavlja izvajanje ukazov, dokler uporabnik emulatorja ne zaustavi. Pri navadni hitrosti primerja število izvedenih T-stanj z realnim časom izvajanja in po potrebi počaka z izvajanjem nadaljnjega ukaza, da se izvajanje približa frekvenci 3,5 MHz. V hitrem načinu tega čakanja ni, zato emulator izvaja ukaze tako hitro, kot dopušča gostiteljski sistem.

Pravilnost implementacije ukazov je bila preverjana tudi s programoma `zexdoc` in `zexall`, različicama preizkuševalnika ukazov Z80, prilagojenima za ZX Spectrum. `zexdoc` preverja dokumentirane zastavice, registre in pomnilnik, `zexall` pa poleg tega preverja tudi nedokumentirani zastavici XF in YF. Programa izvajata skupine ukazov z različnimi začetnimi stanji, iz končnega stanja izračunata kontrolno vsoto CRC in jo primerjata z referenčno vrednostjo, pridobljeno na pravem procesorju Z80. Preizkus ne pove, katera podrobnost ukaza je napačna, temveč samo, da se rezultat razlikuje od referenčnega izvajanja [1] [6].

Poglavje 5

ULA

Procesorja pri branju in pisanju iz pomnilniških lokacij ne zanima s katerim delom pomnilnika dela, pa naj bo to ROM, RAM, ali celo neobstoječ del pomnilnika, ve le, da obstaja 65536 pomnilniških naslovov, na katerih lokacijah lahko obdeluje shranjene podatke. Na popolnoma enak način dela tudi s 65536 V/I naslovi, prek katerih komunicira z zunanjimi napravami (tipkovnica, zvočnik, barvo obrobe zaslona, kasetofon prek vtičnic EAR in MIC, tiskalnik in drugi robni vmesniki in naprave pa prek robnega priključka). [20] [13]

V/I naslovi je sestavljeni iz 16 bitov (A15, A14, ..., A2, A1, A0), ki imajo različne pomene - vrednost 0 katerikoli bita od A0, A1, A2, A3 ali A4 predstavlja indikator specifične operacije, bitov A6 in A7 računalnik ne uporablja, bita A8 in A9 včash podajata dodatne (nedokumentirane) informacije.

Ker v večini primerov višji zlog V/I naslova nima nobenega vpliva na delovanje, je odločilen le nižji zlog V/I naslova, ki označuje V/I vrata (različni V/I naslovi tako mnogokrat naslavljajo ista izmed 256 V/I vrat). [20] [13]

Naslov V/I vrat 254 se nanaša na zvočnik (D4), MIC vtičnico (D3) in nastavlja barvo obrobe zaslona (D2, D1 in D0). Naslov vrat 251 se nanaša na tiskalnik. Naslovi vrat 254, 247 in 239 se nanašajo na dodatne zunanje naprave [20]

Vrednosti na V/I naslovih, ki prehajajo med zunanjimi napravami prek

V/I vrat in CPE so sestavljena sestavljena iz 8 bitov [20] [13]

Kontrolno, podatkovno in naslovno vodilo so (med drugim) dostopna na zadnji strani računalnika. *Str. 118 - prikaz dostopnih povezav do vodil* [20]

5.1 Časovnik

Notranja ura ZX Spectruma je napram ustavljanju programa (2% natančnost) veliko bolj natančna (0,01% natančnost), a se začasno ustavi med spuščanjem zvoka ali operacijami s kasetofonom, tiskalnikom in drugimi zunanjimi napravami. [20]

Hrani se na pomnilniških pozicijah 23674, 23673 in 23672, ki se uporabljajo kot števec 1/50 oz. 1/60 sekunde (odvisno od frekvence osveževanja zaslona) sekunde. [20]

Čas ene urine periode (kremenčevega kristala v CPE) imenujemo stanje T (angl. Transition state), ki traja 0,2857 mikrosekunde, in predstavlja osnovno merilo hitrosti operacij. [13]

5.1.1 Dolžine ukazov in T-stanja

Vsi ukazi so zaporedje osnovnih operacij (strojnih M stanj), sinhronih na urine periode (časovna T stanja).

5.1.2 Implementacija v emulatorju

Časovni model je implementiran v objektu `ULATiming`. Ta hrani skupno število izvedenih T-stanj, trenutno pozicijo znotraj slikovnega okvira in števec prikazanih okvirjev. Emulator uporablja frekvenco procesorja 3,5 MHz in dolžino okvirja 69888 T-stanj, kar ustreza 48K modelu z evropskim video standardom.

Po vsakem izvedenem ukazu procesor časovniku prišteje trajanje ukaza. Ko število T-stanj preseže dolžino okvirja, časovnik poveča števec okvirjev,

označi čakajočo prekinitev in o tem obvesti dele uporabniškega vmesnika, ki so vezani na osveževanje slike. Procesor nato po koncu ukaza preveri, ali je prekinitev omogočena, in jo obdela. Časovnik zato povezuje hitrost izvajanja, periodične prekinitve in osveževanje zaslonskega prikaza.

5.2 Zaslon

V času nastanka računalnika ZX Spectrum so se uporabljali UHF televizorji, kateremu je računalnik oddajal zaslonski signal na UHF kanalu 36. Pomembna je bila tudi različica računalnika, saj so se standardi televizorskih sistemov razlikovali po državah in celinah - evropski in britanski televizorji takrat prikazovali 50 slik na sekundo, sistemi v ZDA, Kanadi in Japonski pa 60 slik na sekundo. Zaradi tega so zanje delali drugačno verzijo računalnika. [21]

5.2.1 Velikost

Televizijski zaslon ima 24 vrstic, vsako dolgo 32 znakov. Vrstice zaslona so oštevilčene od 0 (vrh) do 21, stolpci pa od 0 (leva) do 31. [20]

5.2.2 Razdelitev

Zaslon je razdeljen na dva dela. Zunanji del se imenuje obroba zaslona (angl. border), osrednji del pa papir (angl. paper). Barve teh dveh delov je mogoče poljubno spreminjati, tako neposredno s tipkovnico kot tudi v programu. [21]

Zaslonska slika je razdeljena na 768 položajev (prek 24 vrstic in 32 stolpcev), na katerih se prikazujejo grafični simboli oziroma znaki. Vsak grafični simbol je predstavljen kot 8x8 kvadrat točk, od katerih je vsaka lahko obarvana bodisi v barvo papirja (angl. paper) če ima pripadajoč bit vrednost 0, bodisi v barvo črnila (angl. ink) z vrednostjo bita 1. Cela oblika simbola je tako predstavljena v 8 zlogih (vsak za eno vrstico točk). [20]

A ti niso shranjeni v zaporednih pomnilniških celicah, saj snop elektronov v televiziji potuje po potrebuje te zaporedne informacije posameznega snopa. V pomnilniku so v takem zaporedju vrstic pikslov shranjene najprej vrstice položajev znakov 0 do 7, nato 8 do 15, in nazadnje 16 do 23. [20]

Str. 76 - prikaz delitve zaslona na vrstice in stolpce [20]

Str. 114 - Diagram razbiranja naslovov PRINT položajev in njihove naslove prilaskov (barve papirja in črnila, svetlosti in utripanja) [13]

5.2.3 Nabor znakov

Nabor 256 znakov, ki ga uporablja ZX Spectrum, sestavljajo

- posamezne znake (npr. črke, številke, ločila ipd.)
- znake iz standarda ASCII (razen simbolov £ in ©), in imajo tudi iste kode
- znake, značilne le za ZX Spectrum, kot so grafični simboli (uporabljani v grafičnem načinu delovanja tipkovnice), katerih je prvih 16 sistemsko definiranih, drugih 21 pa uporabniško-definiranih
- žetoni (angl. tokens), ki predstavljajo bodisi kontrolne znake, ki imajo učinek na zunanje naprave (npr. zaslon), bodisi cele besede (npr. PRINT, STOP, ipd.)

[20]

Nabor grafičnih simbolov je prikazan na sliki 5.1.

V ROMu so med naslovoma 3D00 in 3FFF shranjeni točkovni vzorci vseh znakov razen grafičnih. Naslov posameznega znaka, ki ima kod X (npr. 'a' ima kod 61H) izračunamo po formuli $8x + 3C00$ [13]

Povezava med simboli na tipkah in načini vnosa je prikazana na sliki 5.2.

Običajno, ko je nekaj zapisano na položaj znaka, popolnoma uniči tisto, kar je bilo tam zapisano prej. Lahko pa spremembo naredimo tako, da bo novi znak preprosto dodan na vrhu starega. To je lahko še posebej koristno

Symbol	Code	How obtained	Symbol	Code	How obtained
	128	G 8		143	G shift 8
	129	G 1		142	G shift 1
	130	G 2		141	G shift 2
	131	G 3		140	G shift 3
	132	G 4		139	G shift 4
	133	G 5		138	G shift 5
	134	G 6		137	G shift 6
	135	G 7		136	G shift 7

Slika 5.1: Tabela grafičnih simbolov [20].

za pisanje sestavljenih znakov, kot so črke s poudarki na njih, kot v tem programu za tiskanje nemških črk - "ož umlaut nad njim. [20]

5.2.4 Atributi znakov

Vsak znak ima določene sledeče atribute v enem zlogu:

- bit, ki določa svetlost barv, ima dve možni vrednosti: 0 za normalne ali 1 za svetlejše barve svetlo.
- bit, ki določa utripanje, ima dve možni vrednosti: 0 za stabilne barve ali 1 za utripanje, ki se prikaže kot zaporedno zamenjavanje barve črnila in papirja.
- trojica bitov, katerih vrednost določa kodo barve papirja (privzeto bela)
- trojica bitov, katerih vrednost določa kodo barve črnila (privzeto črna)

[21] [20] [13]

ZX Spectrum lahko na (barvnem) zaslonu prikazuje osem barv, označene s številkami od 0 do 7:

MODE	SHIFT										
E	SYMBOL	DEF FN	FN	LINE	OPEN #	CLOSE #	MOVE	ERASE	POINT	CAT	FORMAT
	CAPS	Ink blue	Ink red	Ink magnta	Ink green	Ink cyan	Ink yellow	Ink white	Flash off	Flash on	Ink black
	NONE	Paper blue	Paper red	Paper magnta	Paper green	Paper cyan	Paper yellow	Paper white	Normal brite	Extra brite	Paper black
G	EITHER	◚	◛	◘	◙	◧	◩	◨	■	EXIT GRAPH	DELETE
	NONE	◜	◞	◟	◝	◨	◩	◪	□	EXIT GRAPH	DELETE
K, L or C	CAPS	EDIT	CAPS LOCK	TRUE VIDEO	INV. VIDEO	⇦	⇩	⇧	⇨	GRAPH MODE	DELETE
	SYMBOL	!	@	#	$	%	&	'	(	)	_
	NONE	1	2	3	4	5	6	7	8	9	Ø

Slika 5.2: Tabela simbolov in potrebnih pritiskov tipk zanje [20].

- 0 - črna
- 1 - modra
- 2 - rdeča
- 3 - vijolična ali magenta
- 4 - zelena
- 5 - bledo modra ali cian
- 6 - rumena
- 7 - bela

[21] [20]

Čeprav so barve na prvi pogled razporejene v naključnem vrstnem redu, je tak zaradi dveh razlogov. Prvi razlog se nanaša na uporabo računanika na črno-belih zaslonih, kjer se barve v danem vrstnem redu prikažejo v padajočih odtenkih sivin. Drugi razlog se opira na dejstvo, da lahko človeško oko resnično vidi le tri primarne barve (modro, rdečo in zeleno), druge barve pa so mešanice le-teh. Tak vrstni red barv omogoča "mešanje"barv s seštevanjem

kod primarnih barv - npr. magenta je narejena z mešanjem modre in rdeče barve, zato je njena koda (3) vsota kod za modro (1) in rdečo (2). [21] [20]

Kot barvo črnila ali papirja lahko nastavimo tudi

- vrednost 8, ki označuje "prozorno- barva, s katero jo uporabljate, bo ostala nespremenjena ob izpisu znaka
- vrednost 9, ki pomeni "kontrast- barva (črnilo ali papir), s katero jo uporabljate, bo nastavljena v kontrastu z drugo - bela, če je druga barva temna (črna, modra, rdeča ali magenta) ali črna, če je druga barva svetla (zelena, cianska, rumena ali bela)

[20]

Ker se atributi nanašajo na vseh 64 pik položaja znaka (ne pa na posamezne pike), ne moremo določiti več kot 2 barv ali nastavitev svetlosti in utripanja. Ko nekaj natisnemo na zaslon, poleg vzorca pik spremenimo tudi nastavitve atributov položaja. [20]

5.2.5 Implementacija v emulatorju

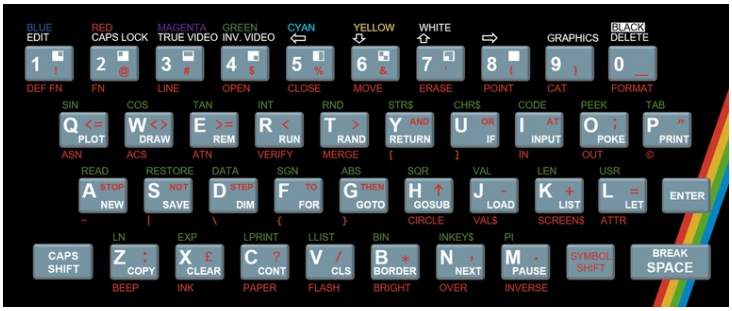
Zaslonski del emulatorja je razdeljen na logiko za razlago pomnilniškega zapisa in na prikaz v uporabniškem vmesniku. Objekt `ULAScreen` vsebuje stalne vrednosti za velikost slike, velikost obrobe, začetne in končne naslove zaslonske datoteke ter datoteke atributov. V njem je tudi preslikava Spectrumovih barv v barve uporabniškega vmesnika ter funkcije za razlago bitov atributnega zloga.

Okno zaslona ob vsakem novem okviru iz pomnilnika prebere območje `0x4000-0x57FF` in območje `0x5800-0x5AFF`. Prvo območje določa vzorec pik, drugo pa barve črnila, papirja, svetlost in utripanje za posamezen položaj znaka. Funkcija za izračun indeksa zloga zaslonske datoteke upošteva nelinearen razpored vrstic, ki ga uporablja ZX Spectrum. Prikaz je narisan na platno Compose, pri čemer se velikost navidezne pike prilagodi velikosti okna, da ostane razmerje slike nespremenjeno.

5.3 Tipkovnica

Tipkovnica Spectruma je zelo podobna tipkovnici navadnega pisalnega stroja - tipke črk in številk so na istih mestih, vendar pa ima vsaka tipka več funkcij. [21]

Razporeditev tipk je prikazana na sliki 5.3.



Slika 5.3: Shema razporeditve tipk tipkovnice ZX Spectruma [12].

Za popravljanje napak uporabimo tipko DELETE. V zgornji vrstici tipkovnice so štiri puščice, ki kažejo v različne smeri, in beseda DELETE. Za uporabo teh tipk moramo pri pritisku uporabiti tipko CAPS SHIFT. Stranske puščice premikajo kurzor v levo ali desno, tipka DELETE pa izbriše znak neposredno pred kurzorjem. [21]

Ko pritisnete ENTER, bo računalnik vaše sporočilo zapisal na vrh zaslona - ali pod tisto, ki ga je zapisal prvič, če je še vedno tam. [21]

Če katero koli tipko pridržimo pritisnjeno več kot tri sekunde, se ta samodejno ponovi, tipka pa takrat oddaja tiho klikajoč zvok. [21] [20]

Znaki, ki jih ZX Spectrum prepozna in uporablja, ne vključujejo le posameznih simbolov (črk, številk itd.), ampak tudi sestavljene znake (ključne besede, imena funkcij itd.). Da bi dobili vse te funkcije in ukaze, imajo nekatere tipke pet ali več različnih pomenov, do katerih pridemo bodisi prek SHIFT tipk (CAPS SHIFT ali SYMBOL SHIFT) bodisi z menjavanjem načinov delovanja tipkovnice. [20]

5.3.1 Načini delovanja

Kurzor je predstavljen kot utripajoča črka, ki označuje tako položaj izpisa naslednjega vnešenega znaka kot tudi trenutni način delovanja tipkovnice prek simbola kurzorja. [21] [20]

Ko Spectrum prvič vklopite, ta po sporočilu o avtorskih pravicah pričakuje vnos ukaza, ki se začne s ključno besedo, katere so natisnjene pod glavnimi simboli na tipkah. Za razliko od večine drugih računalnikov Spectrum omogoča vnos ključnih besed z enim samim pritiskom na tipko. Kurzor s simbolom K (angl. Keyword) predstavlja ta način delovanja za vnos ključnih besed, ki samodejno nadomesti način L, ko naprava pričakuje vnos ukaza ali programske vrstice, kar je odvisno od pozicije kurzorja v vrstici. [21] [20]

V večini drugih primerov bomo v računalnik želeli vnašati besedilo, kar naredimo v načinu delovanja za vnos navadnega besedila. V njem ima kurzor simbol L (angl. Letter). [21] [20]

Na načinu K in L velja, da je za vse kontrolne funkcije, ki so nad tipkami obarvani belo, potrebna pritisnjena tipka CAPS SHIFT, za vse pomožne znake, ki so pod tipkami obarvani rdeče, pa tipka SYMBOLS SHIFT. Če želimo vnesti veliko velikih črk, ne da bi držali tipko CAPS SHIFT, lahko vse črke vnašamo kot velike, tako da najprej pritisnemo tipko CAPS LOCK (CAPS SHIFT in 2). CAPS SHIFT z drugimi tipkami ne vpliva na ključne besede v načinu K, v načinu L pa pretvori male črke v velike. [21] [20]

Način delovanja za vnos besedila z velikimi črkami, v katerem ima kurzor simbol C (angl. Capitals), je različica načina L, v katerem se vse črke prikažejo kot velike. Pritisk na tipko CAPS LOCK povzroči prehod iz načina L v način C ali obratno. [21] [20]

Poleg ključnih besed in besedila (malih in velikih črk ter številk) ima tipkovnica tudi osem vnaprej definiranih grafičnih znakov, ki so na tipkovnici označeni na številčnih tipkah od 1 do 8, ali pa uporabniško-določene grafične znake. Ti se lahko natisnejo na zaslon na podoben način kot črke in številke. Za to je treba tipkovnico preklopiti v način delovanja za vnos grafičnih znakov, kar se naredi s pritiskom tipke GRAPHICS (kombinacija tipk CAPS

SHIFT in 9). Ta način traja, dokler se ponovno ne pritisne bodisi tipka GRAPHICS ali pa le tipka 9. Kurzor bo v tem načinu delovanja označen s simbolom G (angl. Graphics) [21] [20]

Obstaja še en zadnji - razširjeni - način, ki ga označuje kurzor s simbolom E (angl. Extended). Do njega pridemo s hkratnim pritiskom tipk CAPS SHIFT in SYMBOL SHIFT, in traja le do vključno naslednjega pritiska tipke. Ta način omogoča uporabo večine znanstvenih in programskih funkcij, ter pridobitev dodatnih znakov oz. simbolov:

- zgoraj prikazan v zeleni barvi, če med tem ni pritisnjena tipka SHIFT
- spodaj prikazan v rdeči barvi, če je pritisnjena ena od tipk SHIFT
- simbol na številčni tipki, če je pritisnjena tipka SYMBOL SHIFT
- sicer da barvno kontrolno zaporedje

[21] [20]

Kurzor s simbolom vprašaja pomeni, da je pri vnosu ukaza ali znaka prišlo do napake. [21]

5.3.2 Branje stanja tipkovnice

Za branje tipkovnice lahko uporabimo podprograme v ROMu: [13]

KEY SCAN (naslov `0x02BE`) vrne rezultat v registrih D in E:

- če ni pritisnjena nobena tipka, bo vrednost DE enaka FFFF
- če je pritisnjena le ena tipka, bo register D vseboval FF, register E pa bo vseboval kod pritisnjene tipke
- če sta pritisnjeni CAPS SHIFT ali SYMBOL SHIFT in katerakoli druga tipka, bo register D vseboval kod SHIFT tipke, register E pa kod druge pritisnjene tipke

- če sta pritisnjeni tipki CAPS SHIFT in SYMBOL SHIFT istočasno, bo vrednosti DE enaka 2718

in v ničelni zastavici: v zgornjih primerih bo zastavica dvignjena, če pa je naenkrat pritisnjenih več tipk, bo ničelna zastavica ob povratku spuščena, a pri tem spremeni vrednosti registrov A, B, C, D, E, H in L. [13]

Podprogram KEY TEST (naslov 031E) se uporablja za zavržbo rezultate KEY SCAN:

- FFFF (nobena pritisnjena tipka)
- FF27 (pritisnjena le tipka CAPS SHIFT)
- FF18 (pritisnjena le tipka SYMBOL SHIFT)

če je v DE šmiselna vrednost", bo podprogram najprej vrednost registra D bo prenesel v register B ter v register D naložil 0 ,in nadaljeval v eno od dveh smeri:

- če je v DE neveljavna vrednost, bo v register A naložil vrednost registra E (drugi zlog rezultata) in spustil zastavico prenosa
- če je v DE kakršnakoli druga vrednost, bo v register A naložil kod osnovnega znaka tipke in dvignil zastavico prenosa

[13]

(ASCII) kode osnovnih znakov tipk dobimo v BASICu z ukazom CODE, sicer pa v posebni tabeli KEY TABLE na naslovu 0205. [13]

Podprogram KEY CODE (naslov 0333) ugotovi, kateri znak (ne tipka) je bil uporabljen, in naloži njegov ASCII kod v register A:

- register E mora vsebovati kod osnovnega znaka
- register B mora vsebovati FF/27/18
- glede na način delovanja tipkovnice:

  - za način G mora register C vsebovati 02
  - za način E mora register C vsebovati 01
  - za način F mora register C vsebovati 00 in register D 00
  - za način L mora register C vsebovati 00, register D 08 in bit 3 sistemske spremenljivke FLAGS 2 mora biti 0
  - za način C mora register C vsebovati 00, register D 08 in bit 3 sistemske spremenljivke FLAGS 2 mora biti 1

- register C mora vsebovati vrednost sistemske spremenljivke MODE
- register D mora vsebovati vrednost sistemske spremenljivke FLAGS

[13]

Podatke iz tipkovnice, poleg z uporabo podprogramov v ROMu, lahko pridobimo tudi neposredno prek V/I vrat `0xFE`. Tipkovnica je razdeljena v 8 pol-vrstic s 5 tipkami, podatke za pritisnjene tipke lahko zato pridobimo le v petericah tipk naenkrat. [13]

Str. 101 - Skica skupin tipk [13]

Skupino tipk pri tem določimo z vrednostjo registra A ob začetku izvrševanja ukaza (zaporedni bit, ki predstavlja številko željene skupine, mora imeti vrednost 0, vsi ostali pa 1). [13]

Naslov se izračuna kot $254 + 256 * (255 - 2^n)$; $n \in [0, 7]$:

- naslov `0xFEFE` predstavlja tipke med tipkama CAPS SHIFT in V
- naslov `0xFDFE` predstavlja tipke med tipkama A in G
- naslov `0xFBFE` predstavlja tipke med tipkama Q in T
- naslov `0xF7FE` predstavlja tipke med tipkama 1 in 5
- naslov `0xEFFE` predstavlja tipke med tipkama O in 6
- naslov `0xDFFE` predstavlja tipke med tipkama P in 7

- naslov `0xBFFE` predstavlja tipke med tipkama ENTER in H
- naslov `0x7FFE` predstavlja tipke med tipkama SPACE in B

[20] [13]

Register A po izvršenem ukazu branja stanja tipkovnice vsebuje 8-bitni zlog podatkov, pri čemer spodnjih pet bitov (od D0 do D4) označuje pritisnjeno tipko (najnižji bit ustreza najbolj zunanji tipki; za pritisnjeno tipko ima bit vrednost 0, sicer 1). Sicer hkrati lahko, s primerno nastavitvijo registra A pred izvedbo ukaza, beremo tudi več skupin tipk hkrati, a posamezen bit rezultata v tem primeru predstavlja zaporedno pritisnjeno tipko katerekoli izmed izbranih skupin tipk. Sedmi bit (D6) prebranega zloga pa predstavlja vrednost vtičnice EAR. [13] [20]

Str. 121 - Skica tipkovnice Spectruma in kodi tipk (razvrščeni v spirali v nasprotni smeri urinega kazalca) [13]

Ker je neposredno branje stanja tipk precej okorno, zato se marsikdaj programi raje poslužijo zgoraj omenjenega podprograma iz ROMa. [13]

5.3.3 Implementacija v emulatorju

Tipkovnica je v emulatorju implementirana kot matrika osmih pol-vrstic s po petimi biti. Nepritisnjena tipka ima vrednost 1, pritisnjena pa vrednost 0, enako kot pri branju tipkovnice prek V/I vrat na pravem Spectrumu. Objekt `ULAKeyboard` hrani trenutno stanje matrike in pri branju V/I naslova z nižjim zlogom `0xFE` preveri, katere vrstice so izbrane z višjim zlogom naslova. Rezultat je bitni AND izbranih vrstic, dopolnjen z višjimi biti, ki niso vezani na posamezne tipke.

Vhod iz gostiteljskega sistema se najprej prevede v eno ali več tipk matrike. Emulator podpira dva načina: pristni način, kjer fizične tipke približno ustrezajo tipkam Spectruma, in način gostiteljske razporeditve, kjer se znak, ki ga pošlje operacijski sistem, preslika v ustrezno kombinacijo tipk Spectruma. Pri velikih črkah in znakih, ki jih Spectrum vnese s tipko `SYMBOL SHIFT`, se zato v matriki hkrati nastavita tipka za zamik in osnovna

tipka znaka. Ob spremembi načina se vse tipke sprostijo, da v emuliranem računalniku ne ostane navidezno pritisnjena tipka.

5.4 Kasete

ZX Spectrum za trajno hrambo programov uporablja navadne avdio kasete. Program, spremenljivke ali drug del pomnilnika lahko shranimo z ukazom SAVE, kasneje pa ga naložimo z ukazom LOAD. Ukaz VERIFY ne nalaga podatkov, temveč preveri, ali se podatki na kaseti ujemajo s podatki v pomnilniku. Ukaz RUN deluje podobno kot LOAD, le da program po nalaganju tudi zažene. [21] [20]

Pri teh ukazih lahko navedemo ime programa, dolgo največ 10 znakov. Če pri ukazih LOAD, VERIFY ali RUN ime pustimo prazno, Spectrum uporabi prvi ustrezen program, ki ga najde na kaseti. [21] [20]

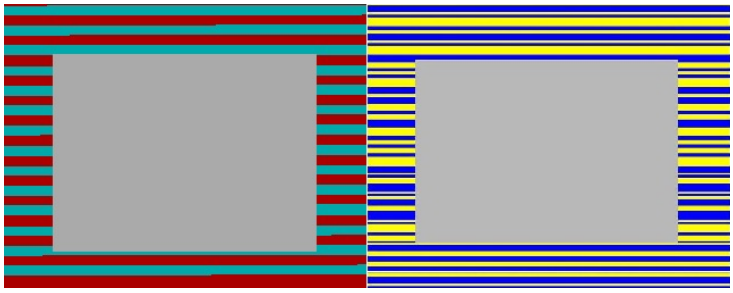
5.4.1 Formati

Kasetne formate je treba ločiti od posnetkov stanja, kot sta formata datotek SNA in Z80. Posnetek stanja hrani le stanje pomnilnika, registrov in drugih indikatorjev stanja sistema, kasetni format pa le hrani podatke kasete v obliki, v kakršni bi jih računalnik prebral iz kasete. S tem se ohrani ne le vrstni red nalaganja blokov, temveč tudi morebitne posebne nalagalnike. [24] [26]

Posnetki kaset v formatih, primernih za uporabo v takšni emulaciji, so razdeljeni na bloke. Pri običajnem nalaganju se najprej prebere glava zapisa z osnovnimi podatki o programu, nato pa še podatkovni bloki z dejansko vsebino. Vsak blok vsebuje zastavični zlog, podatke in kontrolno vsoto. [24]

Format TAP je neposreden zapis takih blokov. Datoteka `.tap` je sestavljena iz zaporedja enako sestavljenih blokov, pri čemer ima vsak blok najprej dva zloga dolžine, nato pa sledijo zastavični zlog, zlog s kontrolno vsoto, in dejanski podatki. Format je zato preprost in primeren za programe, shranjene s standardnimi podprogrami ROMa, ne ohrani pa natančnih časov impulzov, pavz ali delovanja posebnih nalagalnikov. [24]

Vidni potek standardnega nalaganja s kasete je prikazan na sliki 5.4.



Slika 5.4: Pri nalaganju kaset s standardnim podprogramom iz ROMa se branje uvodnega dela bloka na zaslonu kaže kot rdeče in svetlo modre črte, branje dejanskih podatkov pa kot modre in rumene črte. [21] [20] [2]

Format TZX je namenjen čim natančnejšemu hranjenju kaset za ZX Spectrum, predvsem za uporabo v emulatorjih. Datoteka `.tzx` se začne z glavo `ZXTape!` in številko različice, za tem pa sledi en ali več različnih tipov blokov. Če je bil program shranjen z navadnimi ROMovimi rutinami, datoteka v formatu TZX načeloma vsebuje enake standardne podatkovne bloke kot TAP. Njegova prednost je, da lahko poleg osnovnih tipov blokov hrani tudi tudi pavze, časovne podatke o impulzih in druge posebne bloke, ki jih uporabljajo posebni nalagalniki ali zaščite. Zato je TZX primernejši takrat, ko želimo v emulatorju čim bolj natančno ponoviti potek izvirnega nalaganja s kasete. [25] [24]

5.4.2 Implementacija v emulatorju

Nalaganje kaset je v emulatorju razdeljeno na razčlenjevanje datoteke in na izvajanje nalaganja med tekom programa. Razčlenjevanje izvaja objekt `ULATapeParser`. Če se datoteka začne s podpisom `ZXTape!`, jo obravnava kot TZX, sicer kot TAP. Pri formatu TAP zaporedno bere dolžino bloka in surove podatke bloka. Pri formatu TZX prepozna standardne, turbo in čiste podatkovne bloke, druge podprte bloke pa preskoči, kadar ne vplivajo na podatke, ki jih trenutna implementacija potrebuje za nalaganje.

Razčlenjen posnetek kasete je predstavljen z razredoma `SpectrumTapeImage` in `SpectrumTapeDataBlock`. Posamezen blok hrani zastavični zlog, koristne podatke in kontrolno vsoto. Kontrolna vsota se preveri z operacijo XOR čez zastavični zlog in vse podatkovne zloge. Parser pri veljavnih standardnih zapisih označi tudi povezavo med blokom glave in naslednjim podatkovnim blokom, kar kasneje poenostavi izbiro pravega bloka pri zaporednem nalaganju.

Objekt `ULATapeDeck` hrani trenutno vstavljeno kaseto, položaj bralne glave in morebitni podatkovni blok, ki pripada že prebrani glavi. Ko program v emuliranem Spectrumu pokliče ROM rutino `LD-BYTES`, procesor to zazna na naslovu `0x0556` in klic preda kasetnemu podsistemu. Ta iz registrov prebere pričakovani tip bloka, dolžino in ciljni naslov. Pri ukazu `LOAD` se koristni podatki iz izbranega bloka zapišejo v pomnilnik, pri `VERIFY` pa se enak obseg samo prebere in primerja z vsebino bloka. Po koncu se registri nastavijo tako, kot bi se po vrnitvi iz ROM rutine, nato se izvajanje nadaljuje pri skupnem izhodu iz nalagalne rutine.

Tak pristop ne emulira zvočnih impulzov kasete v realnem času, temveč posnema učinek standardnih ROM rutin za nalaganje. Prednost je bistveno hitrejše in zanesljivejše nalaganje programov v razhroščevalniku. Omejitev pa je, da posebni nalagalniki, ki so odvisni od natančnega trajanja impulzov ali od neposrednega branja signala EAR, niso polno podprti.

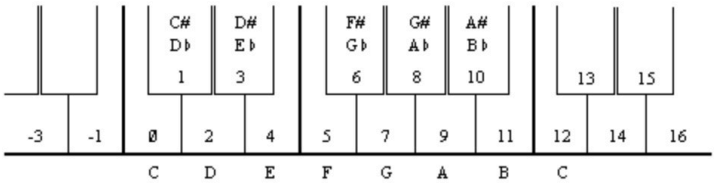
5.5 Zvok

5.5.1 Delovanje in toni

ZX Spectrum lahko proizvaja neskončno raznolike zvoke, pri čemer lahko določimo tako frekvenco kot trajanje tona z ukazom BEEP. [21]

Ukazu podamo dolžino izvajanja tona v sekundah in višino tona v poltonih nad srednjim C. Osrednja frekvenca za ukaz BEEP je srednji C, lahko pa pridobimo katerikoli polton nad ali pod to osrednjo frekvenco. [21]

Kode tonov za ukaz `BEEP` so prikazane na sliki 5.5.



Slika 5.5: Shema tonov in pripadajočih kod tonov [20].

Ker je v računalniku samo en zvočnik, lahko igrate samo en ton naenkrat. Proizveden zvočni signal ne pride le do zvočnika, temveč istočasno tudi do MIC in EAR vtičnic. [20]

Na zvočnik lahko poleg z ukazom BEEP prek ROMa vplivamo tudi z ukazi OUT skozi V/I vrata `0xFE`. Bita 4 v zlogu na vratih določa stanje zvočnika (0 pomeni izključen, 1 pomeni vključen) - frekvenco pa določamo s hitrostjo menjav med stanjema. [13]

5.5.2 Implementacija v emulatorju

Zvočni del je vezan na pisanje v V/I vrata z nižjim zlogom `0xFE`. Vsa pisanja v V/I vrata gredo skozi razred `IOPortSet`; ta shrani zapisano vrednost in jo posreduje objektu `ULABeeper`. Zvočnik uporablja bit 4 zapisane vrednosti, zato se frekvenca zvoka v praksi določi s hitrostjo, s katero program menja ta bit.

`ULABeeper` je v skupni kodi definiran kot pričakovani objekt `expect`, ker je dejansko predvajanje zvoka odvisno od platforme. Na JVM je implementiran z zvočno linijo `SourceDataLine`. Ob vsakem zapisu na vrata `0xFE` primerja trenutno število T-stanj s časom prejšnjega zapisa in iz razlike izračuna, koliko vzorcev mora zapisati v zvočni izhod. Kadar je bit 4 nastavljen, zapisuje vzorce z visoko amplitudo, sicer tišino. S tem se signal zvočnika gradi iz istega časovnega modela, ki ga uporablja procesor.

V spletnih ciljnih platformah je trenutno implementirano le stanje izklopa zvoka, samo predvajanje pa je prazna operacija. To pomeni, da je zvočna

emulacija funkcionalna v namizni različici, v spletni različici pa uporabniški vmesnik ohrani enake kontrole, vendar zvočni izhod še ni izveden.

Poglavje 6

Evalvacija emulatorja

6.0.1 Prikaz delovanja par programov in iger

6.0.2 Primerjava z obstoječimi emulatorji - hitrost, natančnost

Poglavje 7

Zaključek

7.0.1 Evalvacija uspešnosti diplomskega dela (na podlagi ciljev iz uvoda)

7.0.2 Možne izboljšave

Boljša emulacija časovnih ciklov s strani pomnilnika, procesorja in ULA za boljšo sinhronizacijo in emulacijo strojnih napak dejanskega Spectruma: - 1. Memory contention - fetching instructions and other general memory reads/writes - I/O contention - Port `0xFE` (ULA port) contention based on address bus high byte - Snow effect

Raje bi uporabil kaj drugega kot KMP, ker cross-platform funkcionalnosti po mojem mnenju niso uporabne za razhroščevalnik (kar je glavna komponenta tega projekta), prav tako je KMP za desktop privzeto bolj omejen tako v funkcionalnostih kot preformanceu (moral iti nivo nižje od KMP za kaj takega). Tako bi razmišljal o drugih frameworkih, kot so Electron + TS ali Dioxus/Tauri + Rust.

Članki v revijah

[7] Primož Jakopin. “INES”. V: *Moj Mikro* 6 (1985), str. 67.

[11] Ivan Kanič. “INES, sistem za obdelavo besedil, slik in majhnih podatkovnih zbirk”. V: *Moj Mikro* 6 (1984), str. 10.

[16] Jurij Mihelič in Mojca Ferle. “Računalniška emulacija kot strategija digitalnega ohranjanja računalniških sistemov in programske opreme”. V: *Uporabna informatika* 29.4 (2021), str. 196–207. DOI: `10.31449/upinf.133`. URL: `https://uporabna-informatika.si/ui/article/view/133` (pridobljeno 26. 4. 2026).

[18] Iztok Saje. “INES”. V: *Moj Mikro* 4 (1985), str. 16–17.

Literatura

[1] Jeff Avery. *Z80 emulation: Using Z80 Instruction Exerciser (Zexall/Zexdoc)*. 2011. URL: `https://jeffavery.ca/computers/macintosh_z80exerciser.html` (pridobljeno 26. 4. 2026).

[2] Mike Dailly. *How Tape Loading Works*. 2024. URL: `https://lemmings.info/how-tape-loading-works/` (pridobljeno 26. 4. 2026).

[3] Rodney Dale. *The Sinclair Story*. London: Duckworth, 1985.

[4] Guy Fernando. *C Programming on Sinclair ZX Machines*. Created May 2023, last modified Nov 2024. 2023. URL: `https://www.i4cy.com/sinclair/` (pridobljeno 28. 4. 2026).

[5] Fuse emulator project. *Fuse - the Free Unix Spectrum Emulator*. URL: `https://fuse-emulator.sourceforge.net/` (pridobljeno 27. 4. 2026).

[6] Jonathan Graham Harston. *Z80 Instruction Exerciser for the Spectrum*. URL: `https://mdfs.net/Software/Z80/Exerciser/Spectrum/` (pridobljeno 26. 4. 2026).

[7] Primož Jakopin. "INES". V: *Moj Mikro* 6 (1985), str. 67.

[8] Primož Jakopin. *Ines Steve Eva*. 2023. URL: `https://jakopin.net/ines_steve_eva/index_en.php` (pridobljeno 26. 4. 2026).

[9] Primož Jakopin. *O mikroračunalnikih in o INESu*. 2022. URL: https://www.jakopin.net/papers/memoirs/On_microcomputers_and_on_INES_si.php (pridobljeno 26. 4. 2026).

[10] Primož Jakopin. *On STEVE*. 2023. URL: https://www.jakopin.net/papers/memoirs/On_STEVE_en.php (pridobljeno 26. 4. 2026).

[11] Ivan Kanič. "INES, sistem za obdelavo besedil, slik in majhnih podatkovnih zbirk". V: *Moj Mikro* 6 (1984), str. 10.

[12] Magnus Krook. *Home of SoftSpectrum 48*. 2019. URL: https://softspectrum48.weebly.com/ (pridobljeno 26. 4. 2026).

[13] Jaro Lajovic. *Strojni jezik za procesor Z80*. Ljubljana: Mladinska knjiga, 1985.

[14] Sinclair Research Limited. *ZX Spectrum 128 Introduction*. 1st. Sinclair Research Limited. Cambridge, 1986.

[15] Ian Logan in Frank O'Hara. *The Complete Spectrum ROM Disassembly*. 2012. URL: http://www.primrosebank.net/computers/zxspectrum/docs/CompleteSpectrumROMDisassemblyThe.pdf (pridobljeno 26. 4. 2026).

[16] Jurij Mihelič in Mojca Ferle. "Računalniška emulacija kot strategija digitalnega ohranjanja računalniških sistemov in programske opreme". V: *Uporabna informatika* 29.4 (2021), str. 196–207. DOI: 10.31449/upinf.133. URL: https://uporabna-informatika.si/ui/article/view/133 (pridobljeno 26. 4. 2026).

[17] Jonathan Needle. *Spectaculator, Sinclair ZX Spectrum Emulator Home*. URL: https://www.spectaculator.com/spectaculator/ (pridobljeno 27. 4. 2026).

[18] Iztok Saje. "INES". V: *Moj Mikro* 4 (1985), str. 16–17.

[19] SkoolKid. *Spectrum ROM: Memory map*. URL: https://skoolkid.github.io/rom/maps/all.html (pridobljeno 26. 4. 2026).

[20] Steven Vickers in Robin Bradbeer. *ZX Spectrum BASIC Programming*. 3rd. Sinclair Research Limited. Cambridge, 1983.

[21] Steven Vickers in Robin Bradbeer. *ZX Spectrum Introduction*. 1st. Sinclair Research Limited. Cambridge, 1982.

[22] Matt Westcott. *JSSpeccy 3*. URL: https://github.com/gasman/jsspeccy3 (pridobljeno 27. 4. 2026).

[23] Wikipedia contributors. *ZX Spectrum*. URL: https://en.wikipedia.org/wiki/ZX_Spectrum (pridobljeno 28. 4. 2026).

[24] World of Spectrum. *Emulator File Formats*. URL: https://worldofspectrum.org/faq/reference/formats.htm (pridobljeno 22. 4. 2026).

[25] World of Spectrum. *TZX technical specifications*. 2006. URL: https://worldofspectrum.net/TZXformat.html (pridobljeno 22. 4. 2026).

[26] World of Spectrum. *Z80 File Format*. 2004. URL: https://worldofspectrum.org/faq/reference/z80format.htm (pridobljeno 26. 4. 2026).

[27] Sean Young. *The Undocumented Z80 Documented*. Version 0.91. 2005. URL: http://www.z80.info/zip/z80-documented.pdf (pridobljeno 26. 4. 2026).

[28] Zilog. *Z80 CPU User manual*. 2016. URL: https://www.zilog.com/docs/z80/um0080.pdf (pridobljeno 1. 2. 2026).